Introduction to Artificial Intelligence: Machine Learning and Deep Learning

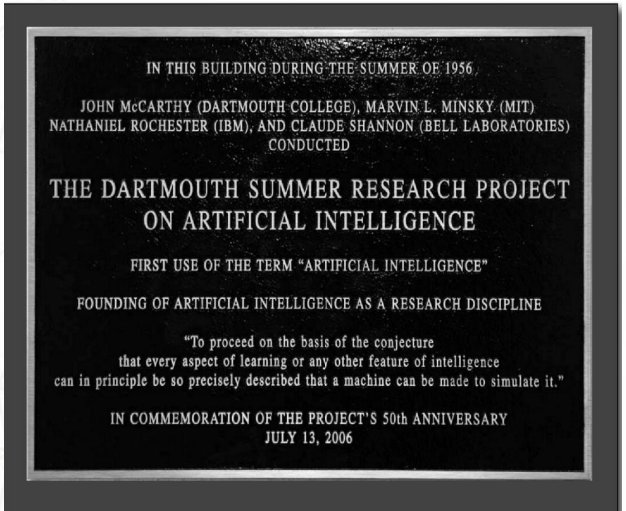
Niloy Ganguly
Professor
IIT Kharagpur

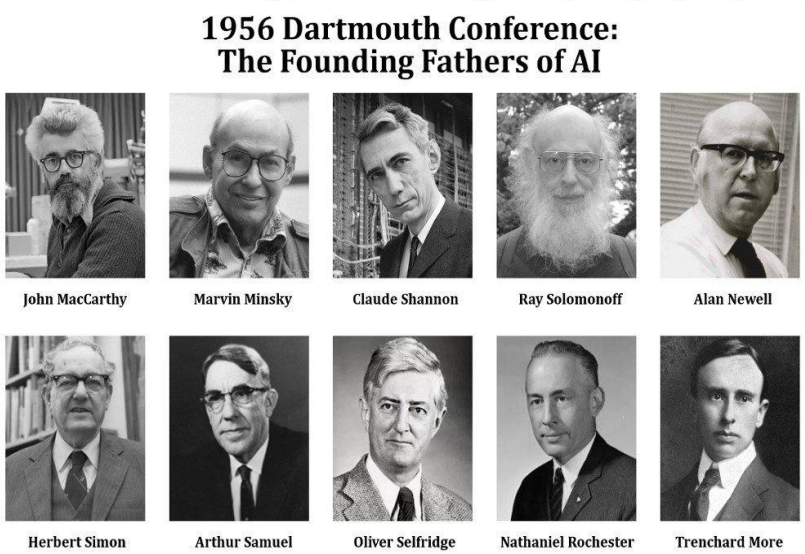




The Birth of AI — 1956 Dartmouth Conference







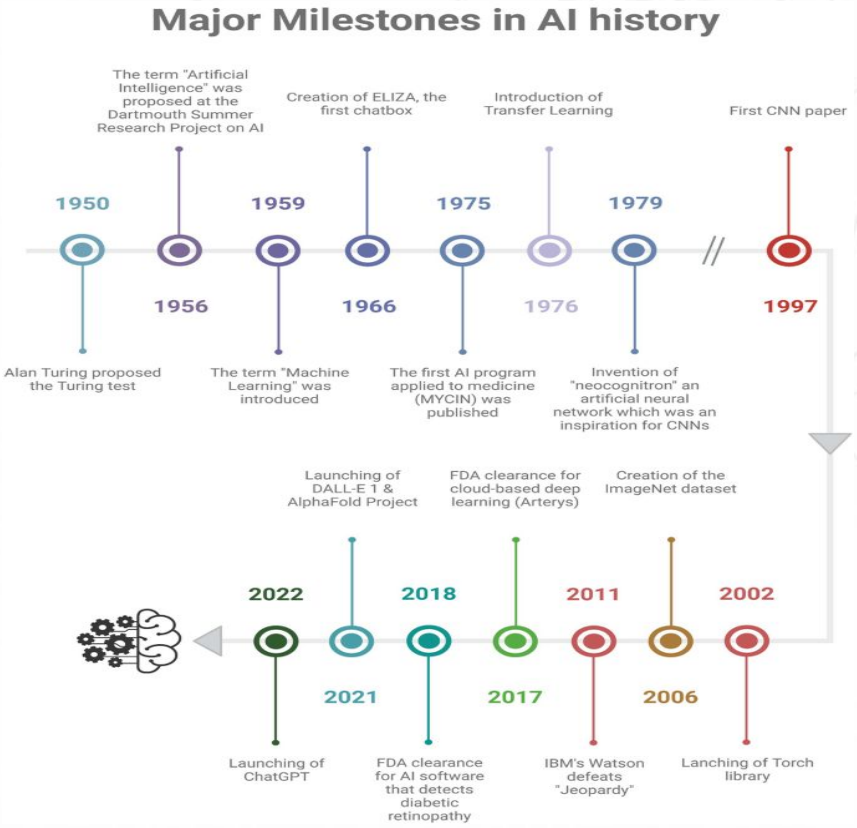


Major Milestones in AI History

Kasparov Proves No Match for Computer

By Rajiv Chandrasekaran
Washington Post Staff Writer
Monday, May 12, 1997; Page A01



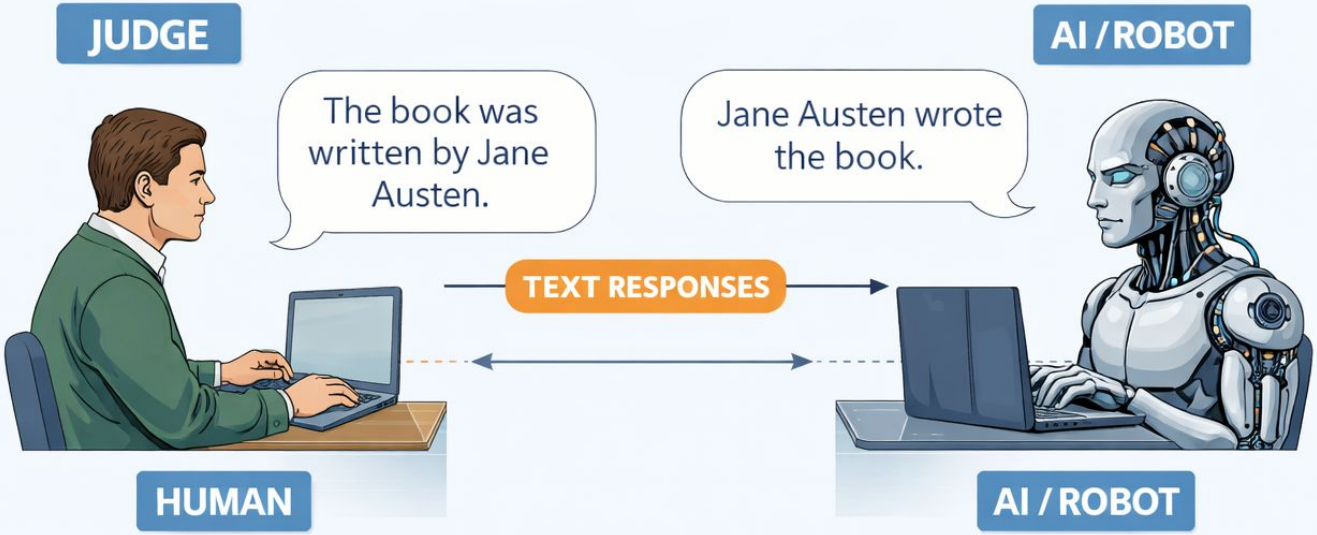


The Turing Test

(Human) Judge communicates with a human and a machine over text-only channel.

* Both human and machine try to act like a human.

*Judge tries to tell which is which.



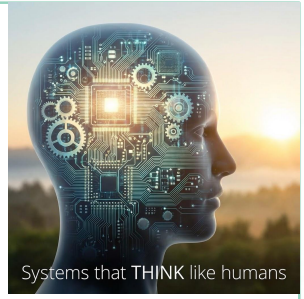
Can the judge tell which is human?

If the Judge fails to differentiate, **AI passes the Turing Test**

Numerous variants - ELIZA program (Weizenbaum 66), Mitsuku (Kuki) (2005 - Present)

What is Artificial Intelligence?

Thinking Humanly / Rationally



- Automation of activities we associate with human thinking (Bellman, 1978)
- Decision making, problem solving, learning
- Study of mental faculties through computational models (Charniak & McDermott, 1985)
- Systems that THINK like humans
- Systems that THINK rationally

Acting Humanly / Rationally

- Systems that ACT like humans (pass Turing Test)
- Systems that ACT rationally (achieve best outcome)
- Four quadrant framework: (Think/Act) × (Human/Rational)
- Modern AI primarily focuses on acting rationally
- Example: AlphaGo acts rationally, not necessarily like a human player



Good Old AI Days

Representing Knowledge



Logic

Formal symbolic reasoning using propositions, predicates, and inference rules.



Rules

If-then production rules that encode expert knowledge for automated inference.



Semantic Graphs / Nets

Graph-based representation of concepts and the relationships between them.

Logic (A Few Statements)

Natural Language	Predicate Logic
Premise All people who are graduating are happy.	$\forall x \quad graduating(x) \rightarrow happy(x)$
Premise All happy people smile.	$\forall x \quad happy(x) \rightarrow smiling(x)$
Fact Someone is graduating.	$\exists x \quad graduating(x)$
Conclusion Is someone smiling?	$\therefore \quad \exists x \quad smiling(x)$ ✓

Rule Based Inference Example

R1 IF

`gas_in_engine AND does_not_start`

THEN **problem(spark_plugs)**

R2 IF

`NOT does_not_start AND NOT lights_on`

THEN **problem(battery)**

R3 IF

`NOT turns_over AND lights_on`

THEN **problem(starter)**

R4 IF

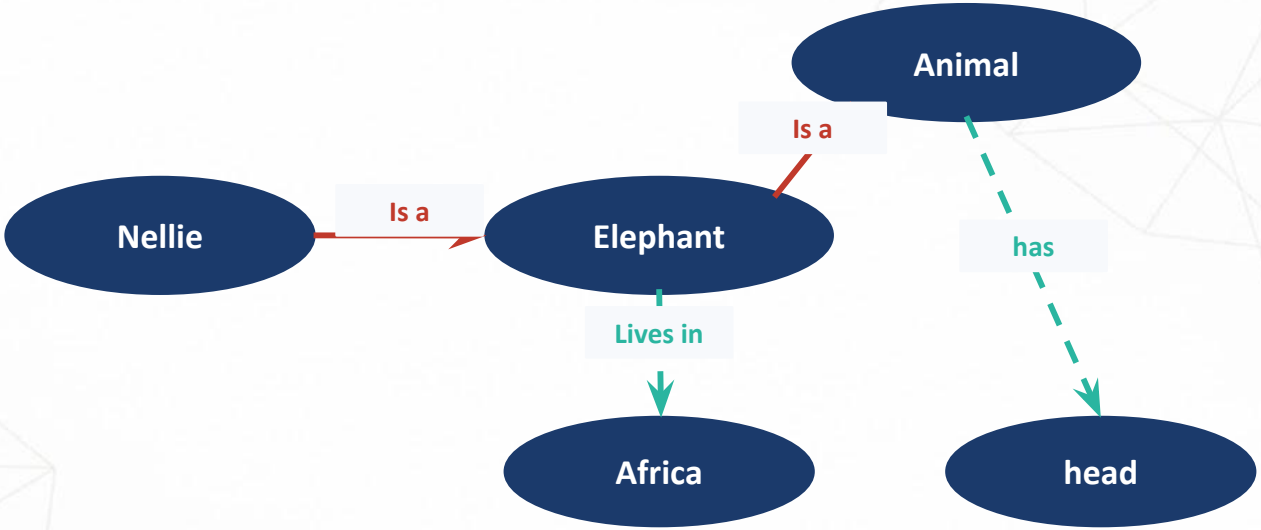
`gas_in_tank AND gas_in_carb`

THEN **gas_in_engine**

Known: gas_in_tank = true | gas_in_carb = true | does_not_start = true →
Hence problem(spark_plugs) ✓

Semantic Nets

A graph where nodes represent concepts and edges represent relationships between them.

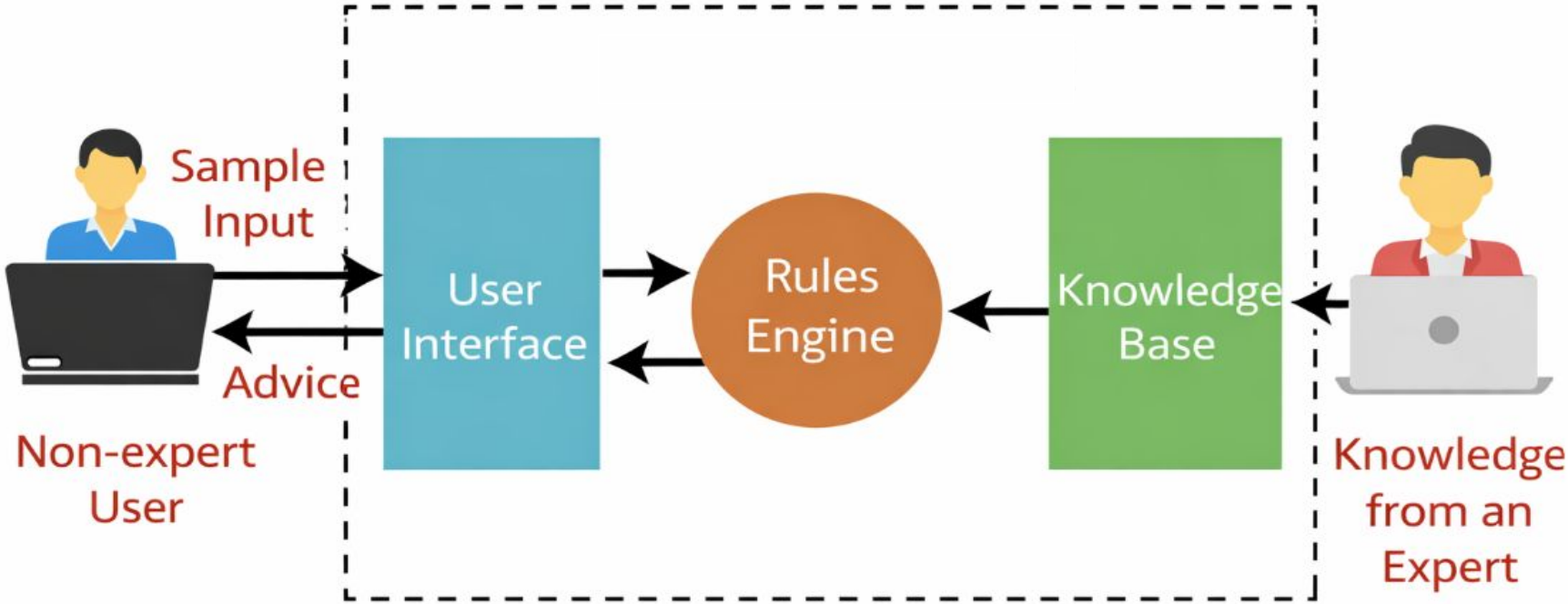


Is-a / Instance-of (inheritance)

Property / Attribute relationship

Expert System

A computer system that emulates the decision-making ability of a human expert



Hit the Wall

Key challenges that caused the first AI Winter — when funding dried up and progress stalled

01 Ambiguity

Highly funded translation programs (Russian to English) were good at syntactic manipulation but failed at semantic disambiguation.

Translation Failure Example

"The spirit is willing but the flesh is weak" → "The vodka is good but the meat is rotten"

02 Scalability & Complexity

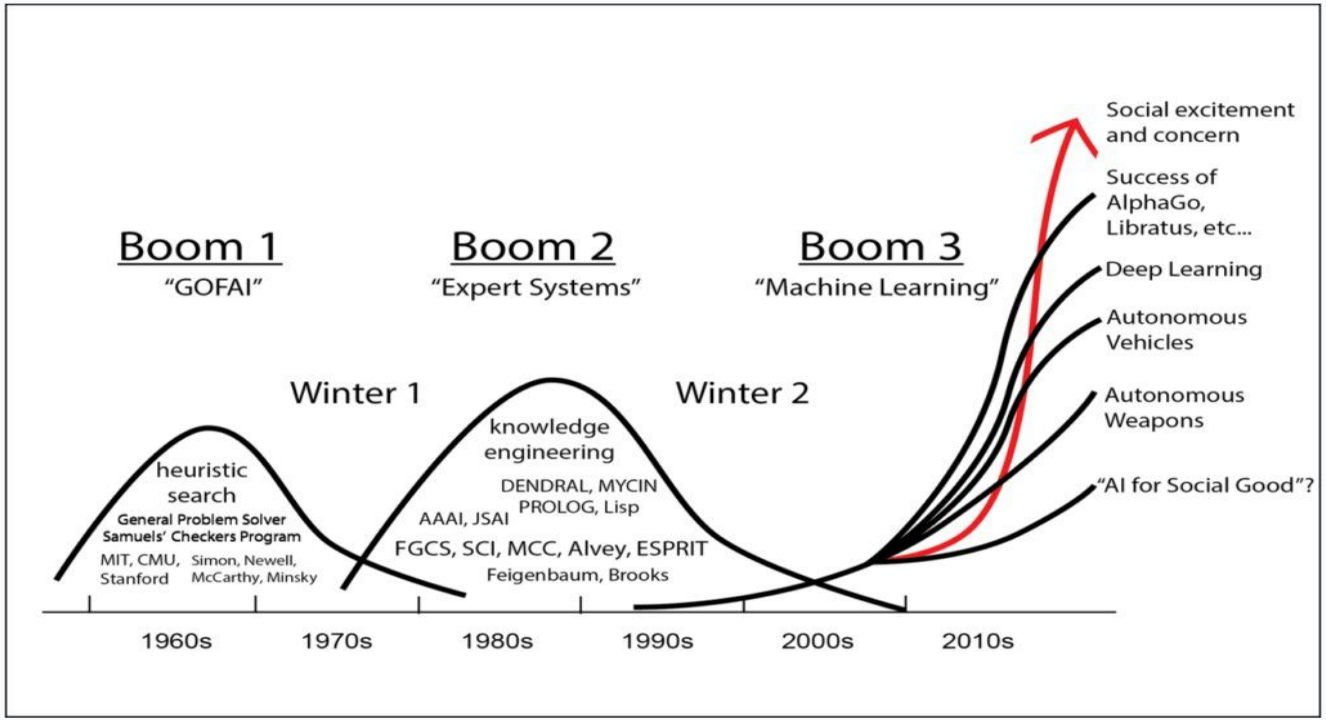
Early AI examples were very small and narrowly scoped. Programs could not scale up to handle bigger or more complex real-world instances.

03 Limitations of Representations

The knowledge representation schemes used (e.g., rules, semantic nets) were too brittle and could not handle uncertainty, ambiguity, or common sense.

AI Winter

Three booms and two winters: the cyclical history of AI research funding and breakthroughs (1960s–2010s)



Key Periods

Boom 1
GOFAI — 1960s

Winter 1
1970s

Boom 2
Expert Systems 80s

Winter 2
1990s

Boom 3
Machine Learning

Machine Learning

Data



Data — The Fuel of Machine Learning

333B+

Emails sent per day

8.5B

Google searches per day

175 ZB

Global datasphere by 2025 (IDC)

90%

World's data created in last 2 years

~1%

Data that is labelled

3 Types

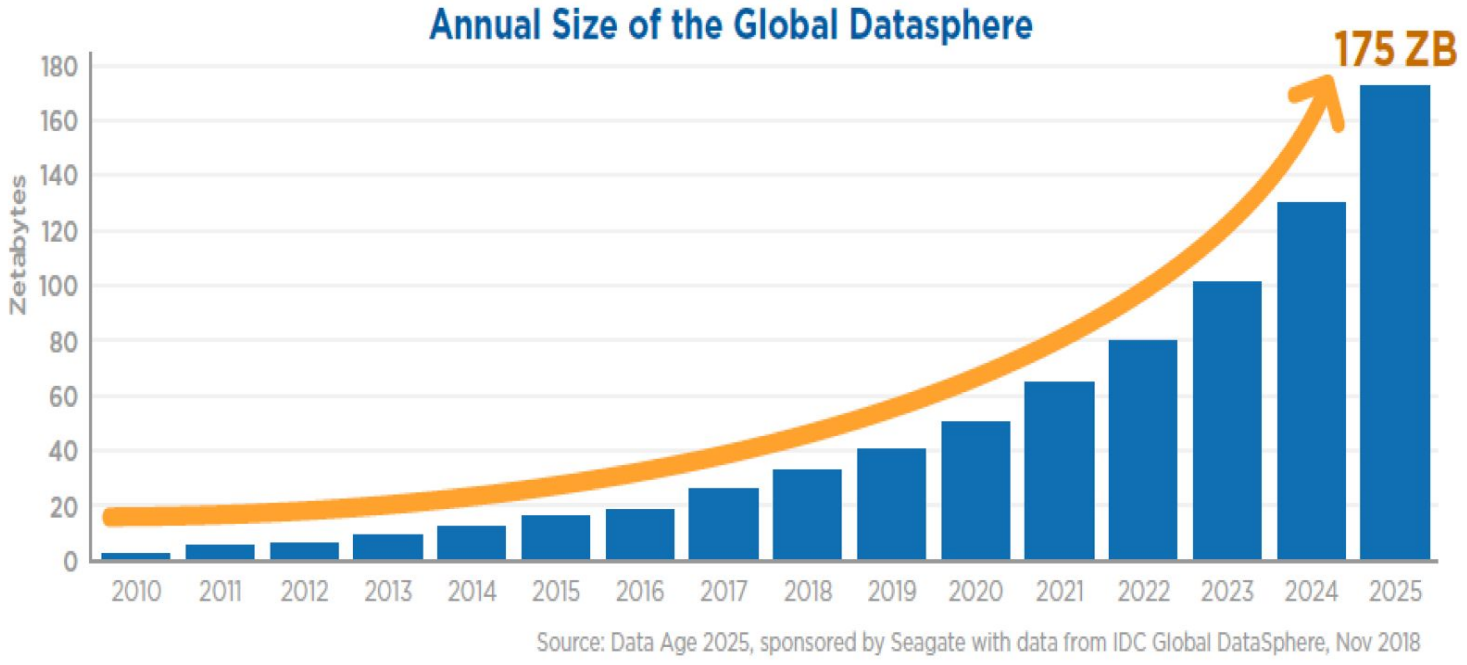
Categorical · Ordinal · Continuous

Data

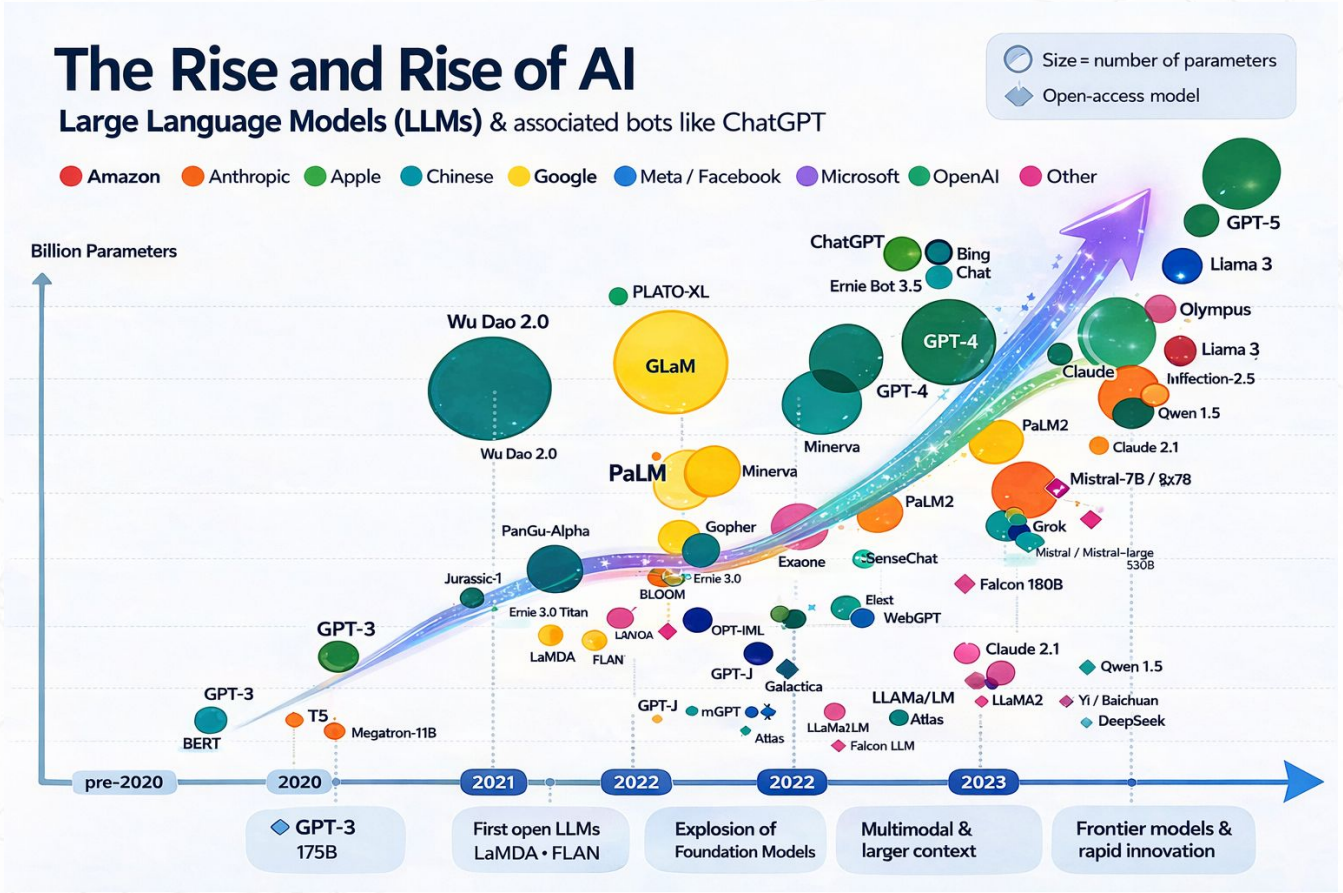
- Collection of measurements or observations that can be used to train a model.
- Can be categorical (cats,dogs,lion,etc), ordinal (tall, medium, short), continuous (10-15,15-20,20-25,...).

Type of Media	Amount per Minute	Amount per Day
Emails sent	231.4 million	333.22 billion
Crypto purchased	90.2 million	129.89 billion
Texts sent	16 million	24.04 billion
Google searches	5.9 million	8.5 billion
Snaps shared on Snapchat	2.43 million	3.5 billion
Pieces of content shared on Facebook	1.7 million	2.45 billion
Swipes on Tinder	1.1 million	1.58 billion
Hours streamed	1 million	1.44 billion
USD spent on Amazon	443,000	637.92 million
USD sent on Venmo	437,600	630.14 million
Tweets shared on Twitter	347,200	499.97 million
Hours spent in Zoom meetings	104,600	150.62 million
USD spent on DoorDash	76,400	110.02 million

Data


Annual Size of the Global Datasphere
175 ZB
Zetabytes
180
160
140
120
100
80
60
40
20
0
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
Source: Data Age 2025, sponsored by Seagate with data from IDC Global DataSphere, Nov 2018

The Journey of Artificial Intelligence



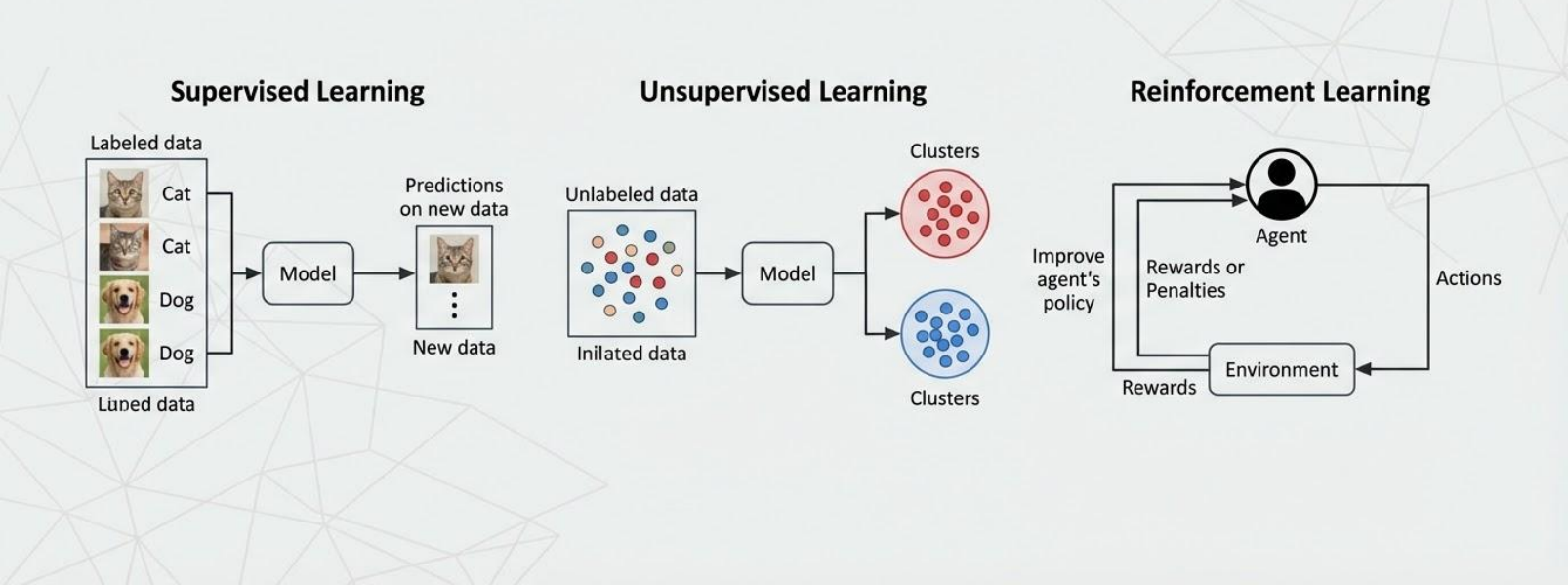
Machine Learning

A computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E.

— Tom Mitchell, 1997

Machine Learning

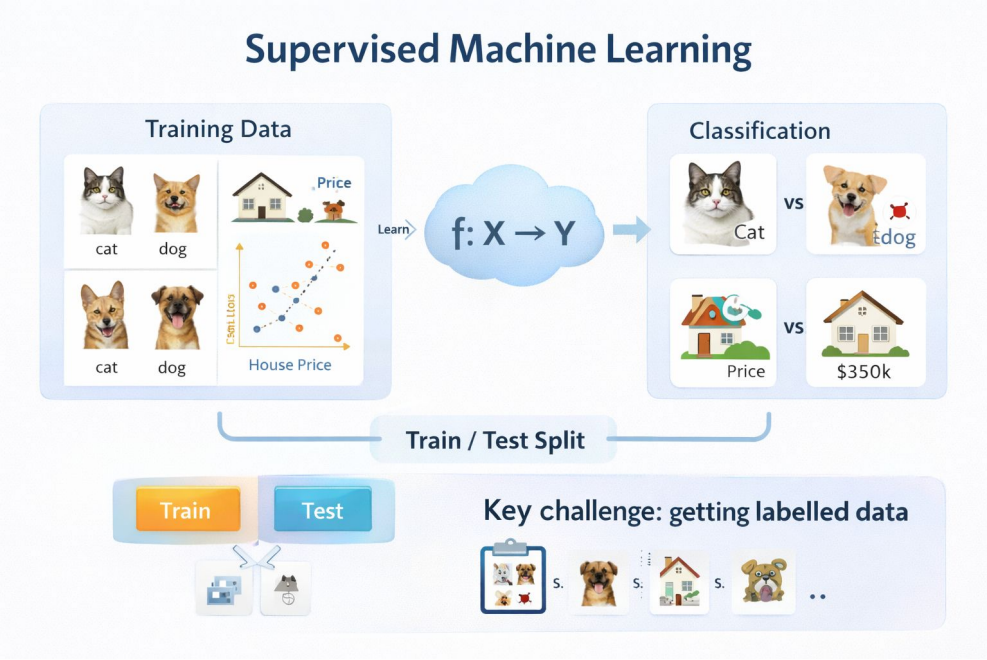
Learning from Data — Supervised, Unsupervised, and Beyond



Supervised Learning — Core Paradigms

Supervised Learning

- Training data has input-output pairs (labels)
- Learn a mapping f: X ⟶ Y
- Classification (cat vs dog) and Regression (house price)
- Train/test split to evaluate generalization
- Key challenge: getting labelled data at scale



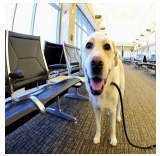
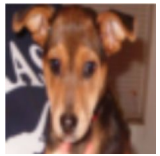
Supervised Learning Workflow (Train Test Split)

- **Procedure :**
  - Divide Dataset into Train and Test

Training Dataset





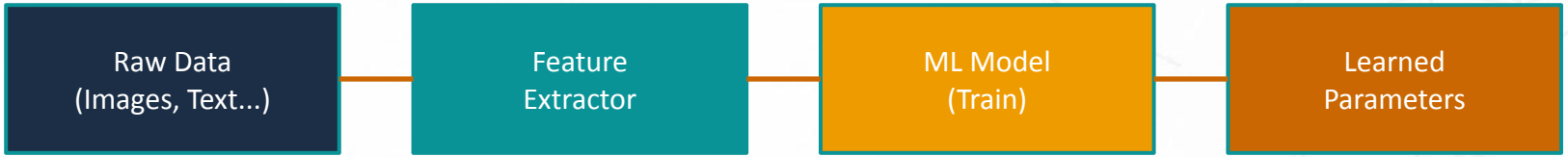



Testing Dataset






Supervised Learning Workflow (Training)

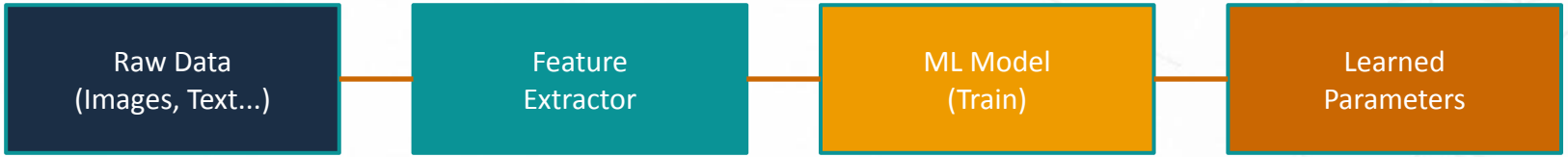




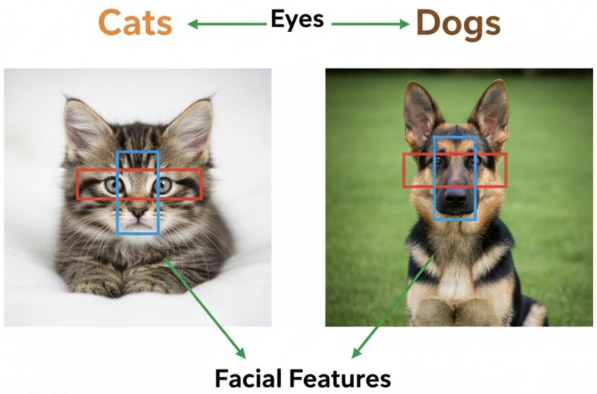
Cat

Dog

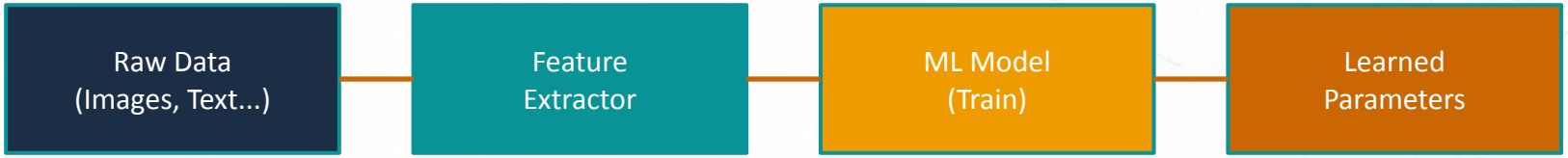
Supervised Learning Workflow (Training)

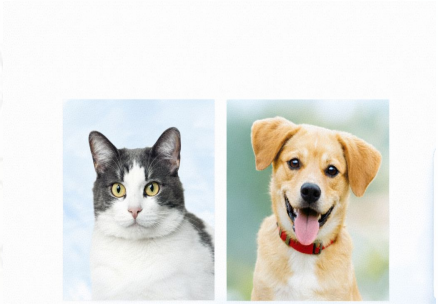


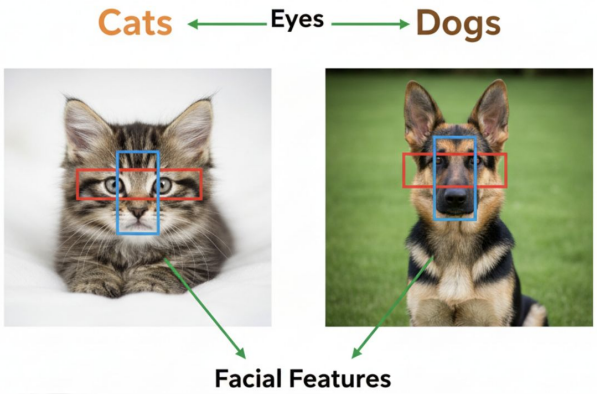


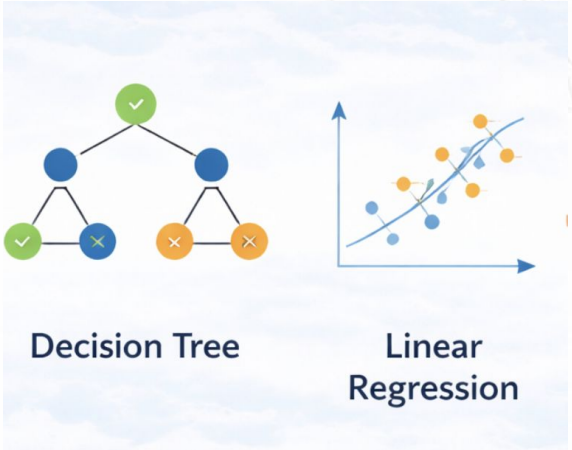


Supervised Learning Workflow (Training)









Supervised Learning Workflow (Testing)

Test Data (Unseen Images, Unseen Text...) → Feature Extractor → Trained ML Model Learned Parameters Fixed, no update → Predicted labels Cat / Dog / class N…Compare to ground truth





Supervised Learning Workflow (Testing)

Test Data (Unseen Images, Unseen Text...)

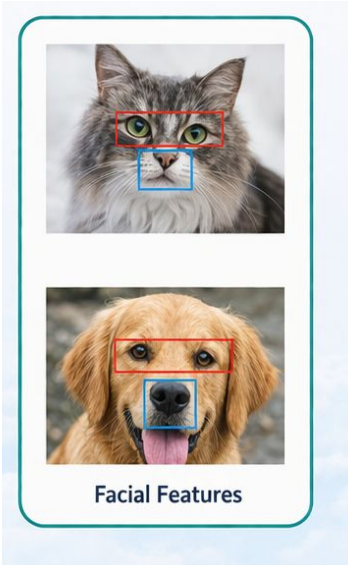
Feature Extractor

Trained ML Model Learned Parameters Fixed, no update

Predicted labels Cat / Dog / class N…Compare to ground truth







Supervised Learning Workflow (Testing)

Test Data (Unseen Images, Unseen Text...)

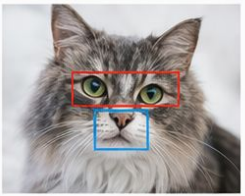
Feature Extractor

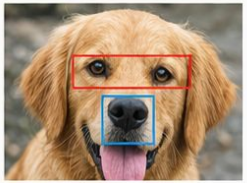
Trained ML Model Learned Parameters Fixed, no update

Predicted labels Cat / Dog / class N…Compare to ground truth

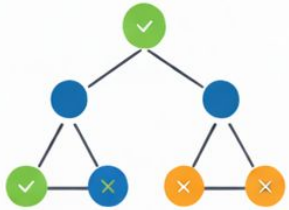


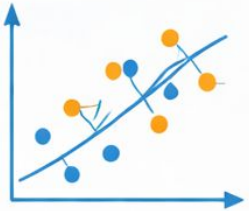






Facial Features





Tutorial on Supervised Learning using scikit-learn

Required python packages-

1. Scikit-learn documentation - https://scikit-learn.org/stable/supervised_learning.html
2. matplotlib - https://matplotlib.org/stable/index.html
3. numpy - https://numpy.org/doc/

```
>>> pip install scikit-learn   # installing the scikit-learn package

>>> pip install numpy          # installing the numpy package

>>> pip install matplotlib     # installing the matplotlib package
```

Scikit-learn

Machine Learning in Python

Supervised Learning

Classification & Regression — SVM, Decision Trees, Random Forests, Gradient Boosting, Linear/Logistic Regression, KNN

Preprocessing & Pipelines

StandardScaler, MinMaxScaler, OneHotEncoder, Imputer, ColumnTransformer, Pipeline API

Unsupervised Learning

Clustering & Dimensionality Reduction — K-Means, DBSCAN, PCA, t-SNE, Gaussian Mixture Models

Key Design Principles

Consistent fit/transform/predict API · NumPy/SciPy compatible · Scikit-learn Estimator protocol · Composable with Pipeline

Model Selection & Evaluation

Cross-validation, Grid/RandomSearchCV, ROC-AUC, Precision-Recall, Confusion Matrix

Ecosystem & Use Cases

Works with pandas, NumPy, Matplotlib · Used in finance, healthcare, NLP, CV · Foundation for AutoML tools

Numpy

Quick Reference — Numerical Python

Import & Setup

import numpy as np · np.__version__ · NumPy is the foundation of scientific Python — arrays, linear algebra, random sampling, and FFTs all live here.

Array Creation

np.array([1,2,3]) · np.zeros((3,4)) · np.ones((2,3)) · np.arange(0,10,2) · np.linspace(0,1,5)

Indexing & Slicing

a[0] · a[1,0] · a[:,1] · a[0:2,:] · a[a>2] boolean mask

Math & Aggregation

a+b a-b a*b · a@b / np.dot(a,b) matrix multiply · np.sum() np.mean() np.std()

Reshape, Stack & Linear Algebra

a.reshape(4,1) a.flatten() a.T · np.hstack / vstack / concatenate · np.linalg.inv() det() eig() norm() · np.linalg.solve(A,b)

Array Attributes & I/O

a.shape a.ndim a.size a.dtype a.itemsize · np.save('arr.npy', a) np.load() · np.savetxt('a.csv', a, delimiter=',') np.loadtxt()

Matplotlib

Quick Reference — Numerical Python

Basic Plots

Line, bar, scatter, histogram, pie — plt.plot() · plt.bar() · plt.scatter() · plt.hist() · plt.pie() · plt.stem() · plt.step()

Figure & Axes

fig, ax = plt.subplots() · fig.set_size_inches() · ax.set_title() · ax.set_xlabel() / ylabel() · ax.legend() · ax.grid()

Styling & Customisation

Colors, markers, line styles · plt.style.use('ggplot') · rcParams · ax.set_xlim() / ylim() · ax.tick_params() · tight_layout()

Subplots & Layouts

plt.subplots(2, 2) · GridSpec · subplot2grid() · fig.add_axes() · sharex / sharey · plt.subplot_mosaic()

Advanced Plot Types

Contour, heatmap, 3D surface · plt.contourf() · ax.imshow() · mpl_toolkits.mplot3d · ax.errorbar() · fill_between()

Saving & Integration

plt.savefig('plot.png', dpi=150) · Jupyter inline · Pandas .plot() API · Seaborn built on Matplotlib · Animation (FuncAnimation)

Supervised Learning using scikit-learn

Problem - Breast cancer prediction is a binary classification problem where the goal is to determine whether a tumor is malignant (cancerous) or benign (non-cancerous) based on measurable characteristics of cell nuclei obtained from medical imaging or biopsies.

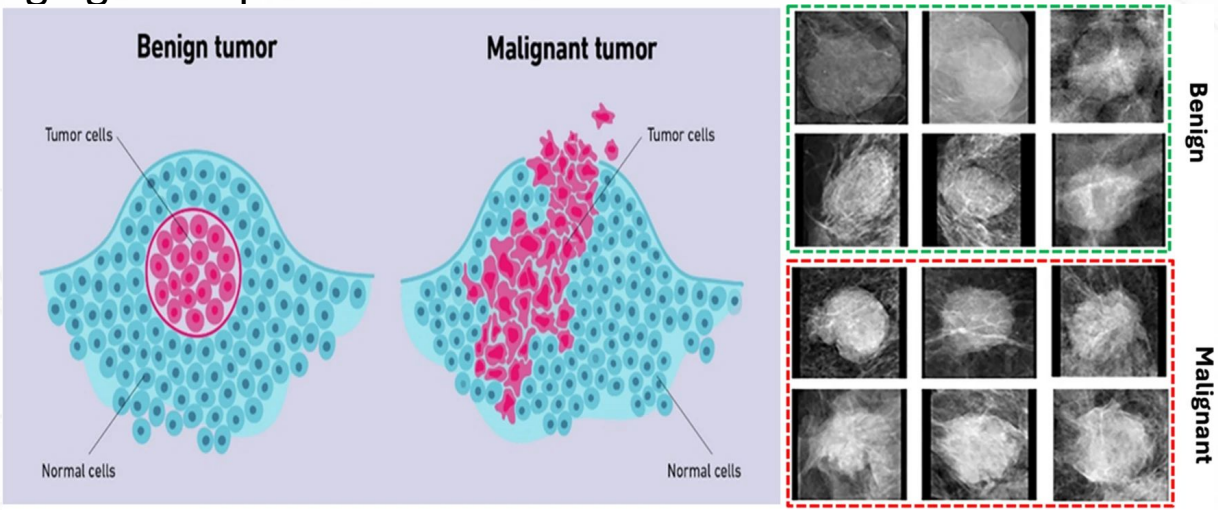


Image Reference: Islam, T., Sheakh, M.A., Tahosin, M.S. et al. Predictive modeling for breast cancer classification in the context of Bangladeshi patients by use of machine learning approach with explainable AI. Sci Rep 14, 8487 (2024). https://doi.org/10.1038/s41598-024-57740-5

Supervised Learning using scikit-learn

Loading the dataset from sklearn datasets.

```
>>> from sklearn.datasets import load_breast_cancer

>>> data = load_breast_cancer()  # Load dataset from sklearn.datasets


>>> list(data.target_names)  # Check class names

[np.str_('malignant'), np.str_('benign')]


>>> X = data.data

>>> y = data.target  # benign (non-cancerous) malignant (cancerous)
```

Print some target labels (ground truth)

Supervised Learning using scikit-learn

Dataset train-test split

```
>>> from sklearn.model_selection import train_test_split
```

```
>>> X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Split the dataset into training and testing sets

- Split data into training set (learn patterns) and test set (evaluate performance)
- Prevents overfitting
- Standard split: 80-20

Supervised Learning using scikit-learn

Feature Scaling

```
>>> from sklearn.preprocessing import StandardScaler  # Import StandardScaler
```

```
>>> scaler = StandardScaler()  # Instantiate the scaler object

>>> X_train = scaler.fit_transform(X_train)
>>> X_test = scaler.transform(X_test)
```

- Logistic regression is scale-sensitive
- Features should be normalized
- Improves convergence & performance

Standard Scaler

Standardization: $z = \frac{x - \mu}{\sigma}$

Mean: $\mu = \frac{1}{N}\sum_{i=1}^{N} x_i$

Standard Deviation: $\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}$

Min-Max scaling: $X_{\text{norm}} = \frac{X - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}}$

Supervised Learning using scikit-learn

Model Training

```
>>> from sklearn.linear_model import LogisticRegression

>>> model = LogisticRegression(max_iter=1000)    # Initialize the Logistic Regression model


>>> model.fit(X_train, y_train)    # X_train → input features, # y_train → corresponding labels (0 = malignant, 1 = benign)
```

Supervised Learning using scikit-learn

Model Testing

```
>>> y_pred = model.predict(X_test)  # Import evaluation metrics

>>> from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

>>> print("Accuracy:", accuracy_score(y_test, y_pred))  # Overall percentage of correct predictions

>>> print("\nConfusion Matrix:") # Display confusion matrix header
>>> print(confusion_matrix(y_test, y_pred)) # Show TP, TN, FP, FN counts

>>> print("\nClassification Report:") # Display detailed metrics header
>>> print(classification_report(y_test, y_pred)) # Show precision, recall, F1-score, support
```

Evaluation Metrics (Confusion Matrix)

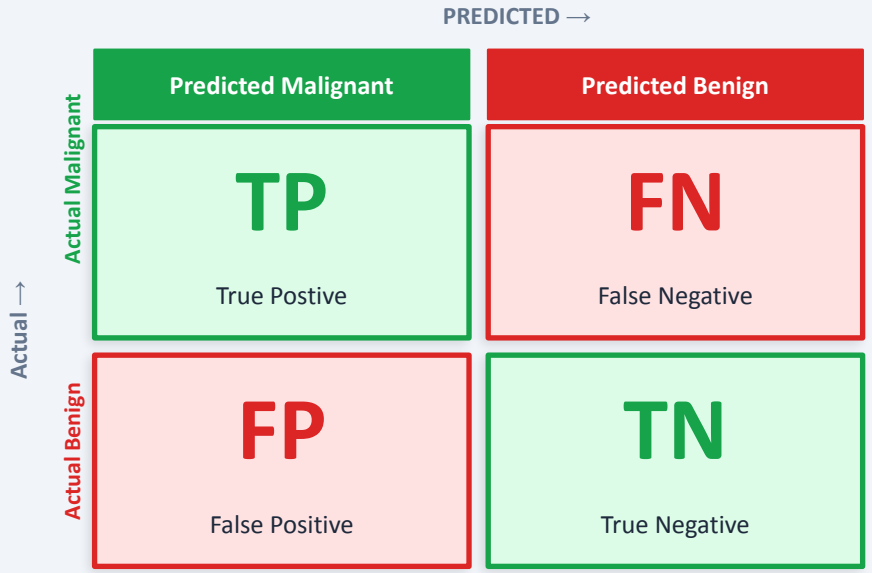
Confusion Matrix — What is it?

A structured table that compares a model's predictions against the actual ground-truth labels — revealing exactly where it gets things right and where it goes wrong.

- **Rows** = Actual (true) class
- **Columns** = Predicted class
- **Each cell** = Sample count for that outcome

Used in: cancer diagnosis, spam filtering, fraud detection.

2×2 Confusion Matrix



Accuracy, Precision, Recall, F1 Score

Accuracy

(TP + TN) / Total

Overall correct predictions out of all samples.

When to use

Best for balanced datasets where class sizes are roughly equal.

Precision

TP / (TP + FP)

Of all Positive predictions, how many were actually correct?

When to use

Use when false alarms are costly — e.g. spam, fraud detection.

Recall

TP / (TP + FN)

Of all actual Positives, how many did the model catch?

When to use

Use when missing a positive is dangerous — e.g. disease screening.

F1 Score

2 × (P × R) / (P + R)

Harmonic mean of Precision and Recall.

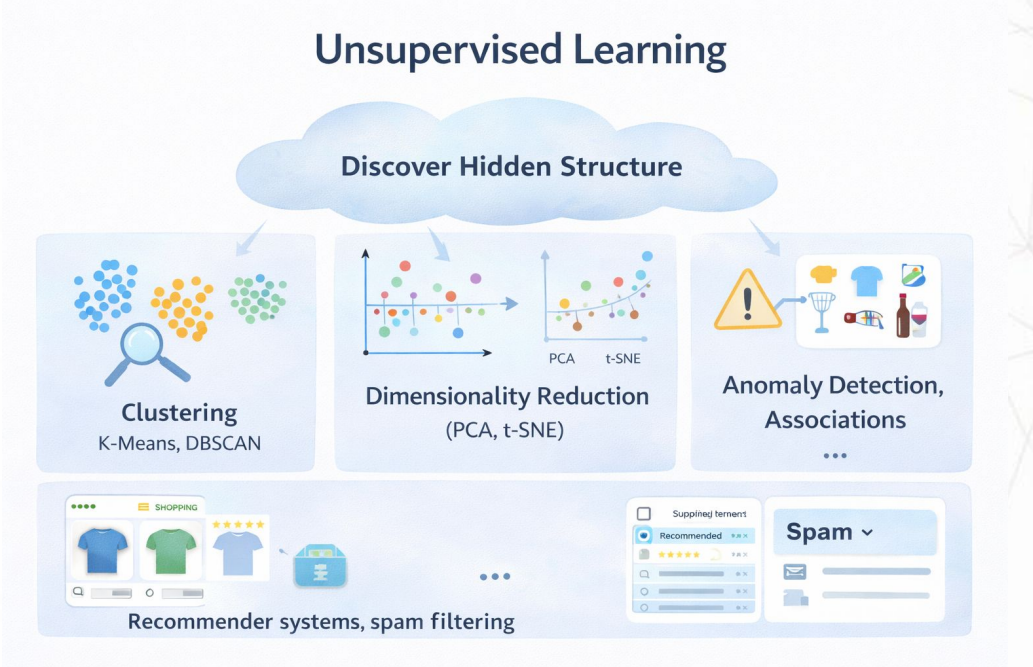
When to use

Best single metric for imbalanced datasets where both errors matter.

Unsupervised Learning — Core Paradigms

Unsupervised Learning

- No labels — discover hidden structure
- Clustering: group similar items (K-Means, DBSCAN)
- Dimensionality Reduction (PCA, t-SNE)
- Anomaly Detection, Association Rules
- Example: Recommender systems, spam filtering



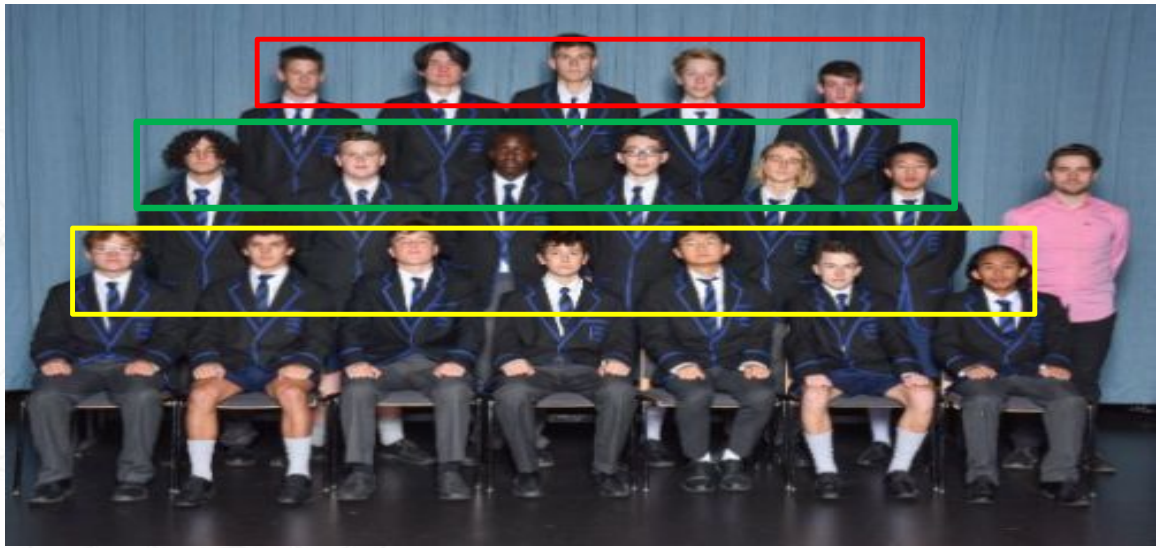
Unsupervised Learning

- **Scenario [Human Behavior]**
  - Suppose your class is being ready for a group photograph and taking positions in order of heights.
  - You do not possess any prior knowledge of the heights of your classmates.



Unsupervised Learning

- **Scenario [Human Behavior]**
  - You try to get into a position according to the height.
  - You figure it out without being told as to where to stand.



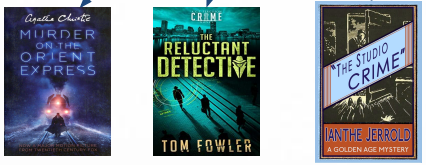
Unsupervised Learning



Those who bought this



also bought these



Recommender Systems
Example of Unsupervised Learning

Why do we need to compress high-dimensional data?

Motivation for PCA

😵 The Problem

Real data lives in very **high dimensions**.

- **Images:** 224×224×3 = 150k dims
- **Text:** vocab size ~100k
- **Genomics:** 20k+ genes

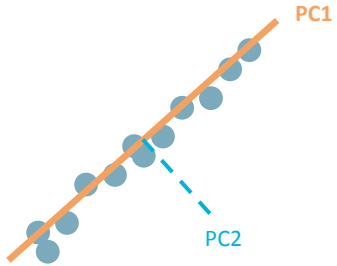
⚠️ Curse of Dimensionality
Distance loses meaning in high-d

💡 The Insight

Data has structure, **not all directions matter equally**

Most variation lives in a **low-dimensional subspace**

Find axes of **max variance**, project:



✅ What PCA Does

Finds an **orthogonal basis** that maximises variance

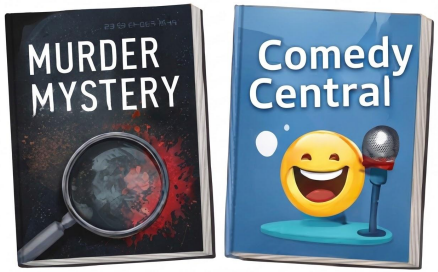
- No labels needed
- Keep top-k components
- Discard noise/redundancy

d = 1000 → k=2

compress while keeping structure

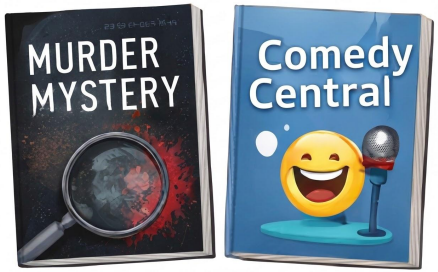
Unsupervised Learning Workflow (Training)

Raw Data (Images, Text...) → Feature Extractor → Dimensionality Reduction → Clustering



Unsupervised Learning Workflow (Training)

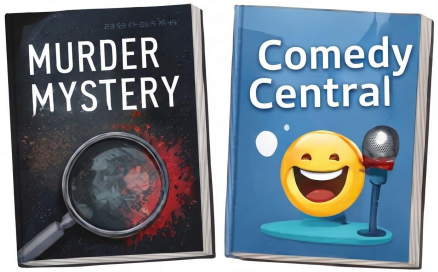
Raw Data (Images, Text...) → Feature Extractor → Dimensionality Reduction → Clustering



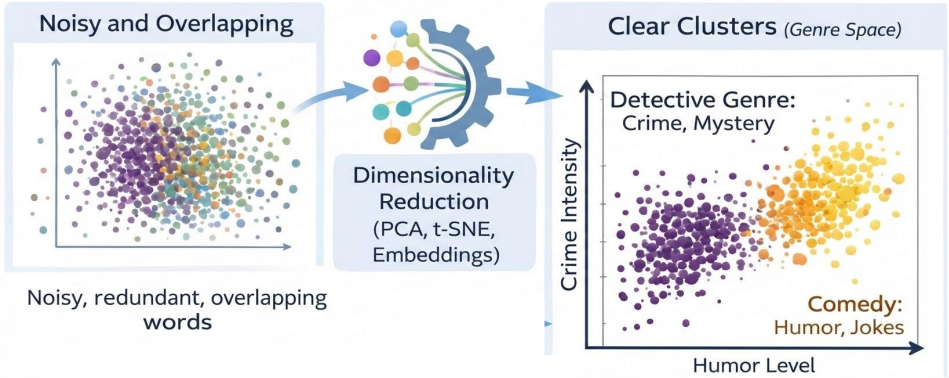


Unsupervised Learning Workflow (Training)

Raw Data (Images, Text...) → Feature Extractor → Dimensionality Reduction → Clustering





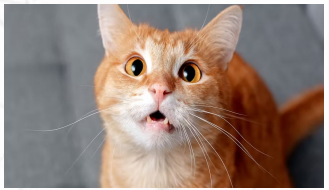


Challenges and Limitations

Supervised Learning requires labelled data.

Large amounts of data available (in Zettabytes).

Almost all are unlabelled.





The Feature Extraction Bottleneck

Raw Data (Images, Text...) → Feature Extractor → Supervised / Unsupervised Model → Predictions / Clusters

⚠ The Problem: Hand-crafted Features

Supervised Learning

- Domain experts manually define features (e.g., facial landmarks, TF-IDF weights)
- Requires labelled data **and** expert feature design for every new

Unsupervised Learning

- Hand-crafted features (word counts, PCA axes) rely on heuristics chosen by experts
- Does not generalise across domains without manual redesign

Costly, slow & brittle — a new domain needs a new expert.

✨ The Solution: Learned Embeddings

Automatic — the model learns which features matter directly from raw data, no human labelling of features required

Transferable — one embedding model generalises across tasks (classification, clustering, search) without redesign

Dense & Scalable — compact low-dimensional vectors capture rich semantics; geometry encodes meaning

The only scalable way forward: let the model automatically learn feature vectors — Embeddings.

Feature Learning through Embeddings

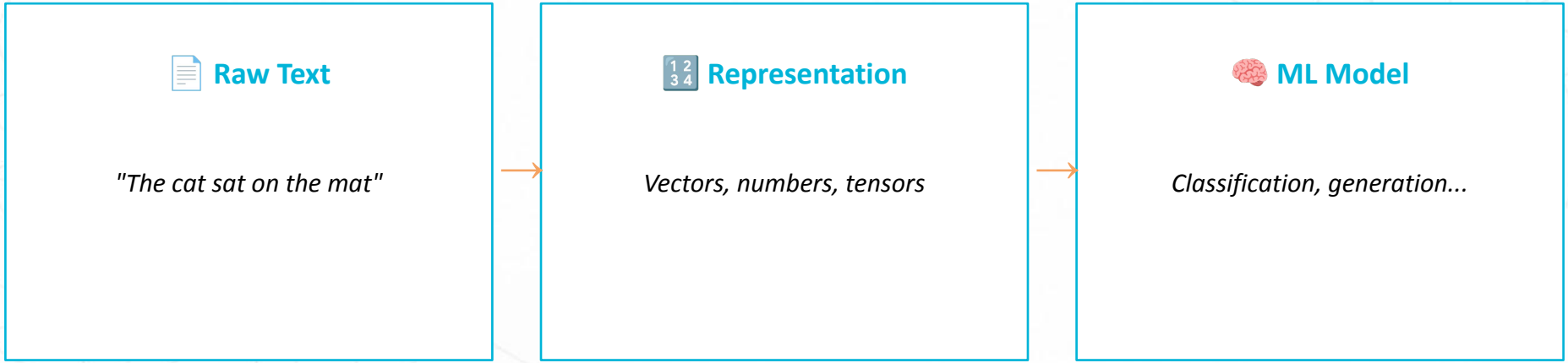


Embeddings are powerful tools for representing **complex data** in a **compact form** while **preserving semantic meaning**.

Feature Learning through Embeddings

A motivating journey through text representations from hand-crafted statistics to learned dense vectors

How do we turn raw text into something a machine can learn from?



The representation step is the key bottleneck — and the biggest opportunity.

One-Hot Vector Representation

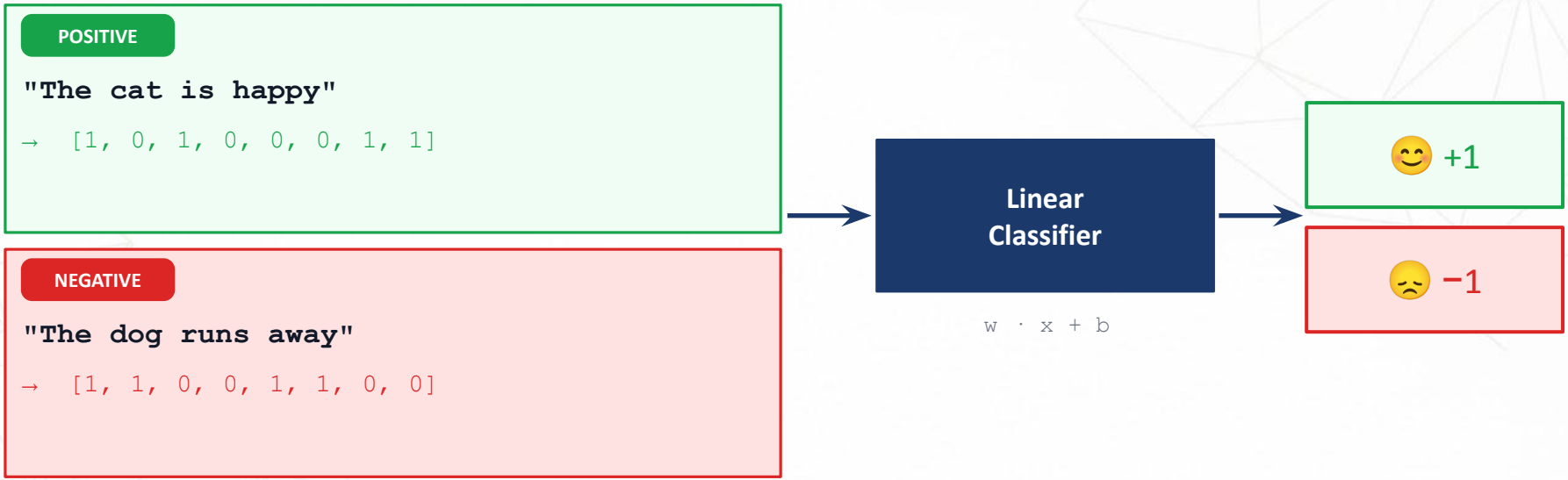
Step 1: Assign every word a unique index in a fixed vocabulary **Vocabulary (|V| = 8)**

	0 cat	1 dog	2 king	3 queen	4 apple	5 runs	6 happy	7 the
cat →	1	0	0	0	0	0	0	0
king →	0	0	1	0	0	0	0	0
queen →	0	0	0	1	0	0	0	0

Key property: Each word is a point in high-dimensional space. The '1' is at index i, everything else is 0.

Classification with One-Hot Vectors

Example: sentiment classification — can a linear model separate positive from negative? **Bag-of-Words Representation**



✅ What works

- Simple, fast, interpretable weights
- Works for word-frequency signals

⚠️ What struggles

- "happy" and "joyful" treated as unrelated
- Sparse vectors → huge, mostly-zero inputs

Limitations of Sparse Representations

🔤 No Semantic Meaning

"king" and "queen" are just two unrelated dimensions

`"cat" ≠ "kitten" in TF-IDF`

📏 Curse of Dimensionality

Vocabulary of 100k words = 100k-dim sparse vectors

`99%+ entries are zero`

🔀 No Word Order

"dog bites man" and "man bites dog" are the same

`Bag-of-words assumption`

🆕 Out-of-Vocabulary

New words at test time create unknown slots

`Zero vector = no info`

→ These are fundamental flaws. We need a smarter representation.

Why Sparsity Breaks Similarity

The dot product between any two one-hot vectors is always zero

Cosine Similarity between one-hot vectors:

cat	1	0	0	0	0	0	0	0	•	kitten	0	0	0	0	1	0	0	0	=	0.00
king	0	0	1	0	0	0	0	0	•	queen	0	0	0	1	0	0	0	0	=	0.00
happy	0	0	0	0	0	0	1	0	•	dog	0	1	0	0	0	0	0	0	=	0.00

💡 **Insight**

One-hot vectors are orthogonal by construction — the model can never discover that "cat" and "kitten" are related.
We need a representation where similar words are geometrically close.

Distributional Semantics — Words by Context

"You shall know a word by the company it keeps" — J. R. Firth, 1957

The Big Idea

① **A word's meaning** is defined by the words that frequently appear around it.

② **Context window:** look at ±k words on either side of the target word.

③ **Build a vector** from the distribution of context words across a large corpus.

④ **Similar contexts** → similar meaning → similar vectors.

Context Window (k = 2)

…government debt problems turning into **banking** crises as happened in 2009…

…saying that Europe needs unified **banking** regulation to replace the hodgepodge…

…India has just given its **banking** system a shot in the arm…

These context words will represent banking

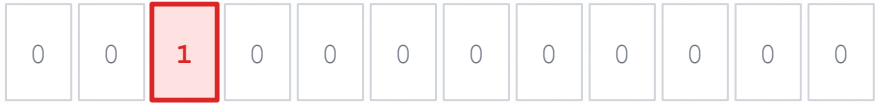
💡 **Key shift:** Instead of a word's position in a fixed vocabulary, we encode what words it co-occurs with.

Count co-occurrences → compress → dense vector. This is the road to word embeddings.

Dense Representations — Distributed Semantics

Replace one sparse index with a short, dense vector where every dimension carries meaning

SPARSE (One-Hot)



… (50,000 dims)

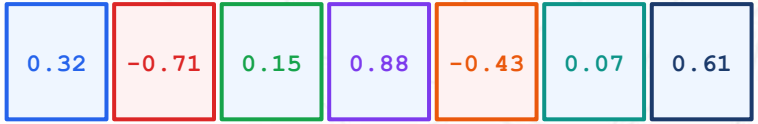
✗ 99%+ zeros

✗ No similarity

✗ Huge memory

learn weights →

DENSE (Word Embedding)



(300 dims — every value carries meaning)

✓ All values active

✓ Geometry = meaning

✓ Compact & fast

Geometry encodes meaning — famous examples:

```
vec("king") - vec("man") + vec("woman")   ≈ vec("queen")
vec("Paris") - vec("France") + vec("Germany")   ≈ vec("Berlin")
cosine( vec("cat"), vec("kitten") )   ≈ 0.92  ✓
```

Feature Learning through Embeddings

The embedding model learns dense vectors from raw data — no hand-crafting required.

What are Embeddings?

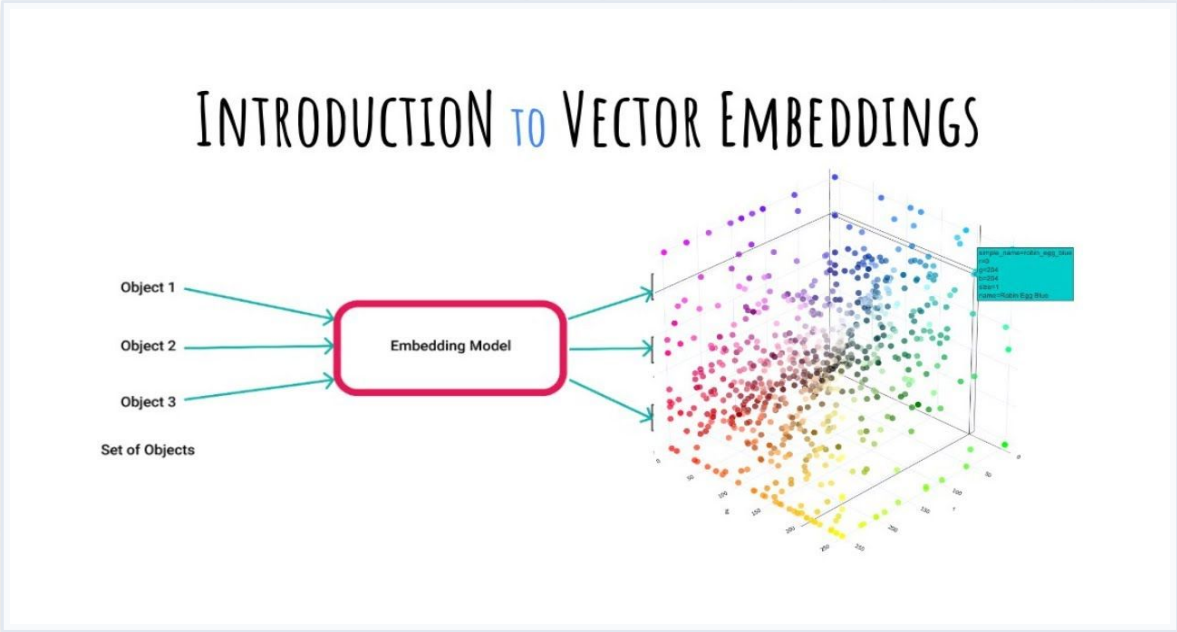
Dense numerical vectors representing objects (words, images, users) in a high-dimensional space where similar items cluster together.

How the Model Learns

Objects feed into the Embedding Model which outputs a fixed-length dense vector capturing each object's semantic relationships.

Why Embeddings Win

No manual feature engineering needed. Captures latent structure. Enables similarity search, clustering & transfer learning.



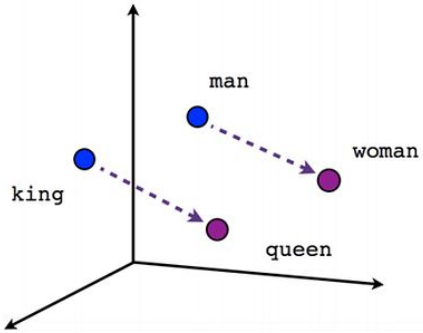
Input
Any raw object: word, image, user ID

Model
Neural network trained end-to-end on a task

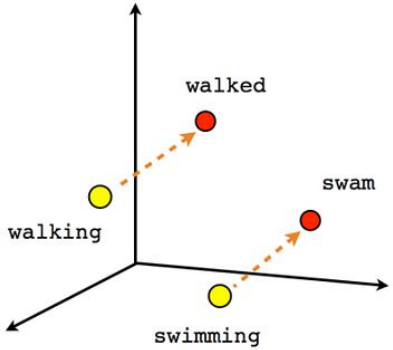
Output
Dense fixed-length vector for every object

Result
Similar objects sit nearby in vector space

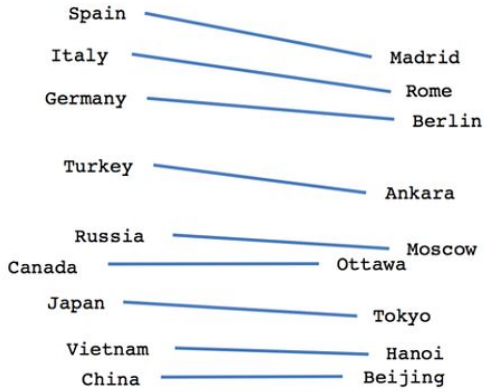
Embeddings



Male-Female



Verb tense



Country-Capital

king - man + woman = queen

Deep Learning - End-to-end Learning

Deep Learning

Feature Learning: Neural networks learn complex features from simple ones through multiple layers.

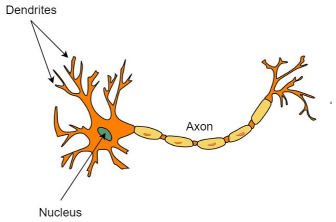
No Manual Feature Selection Needed: Models automatically identify the most relevant features from raw data.

Practical Applications: Effective in areas such as computer vision and natural language processing.

Biological Inspiration — The Neuron

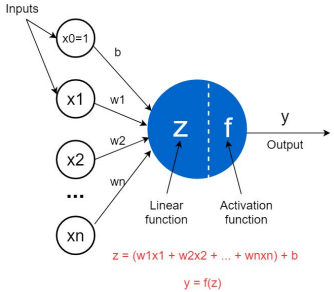
Biological Neuron

- ~86 billion neurons in the human brain
- Dendrites: receive signals from other neurons
- Soma (cell body): integrates incoming signals
- Axon: transmits output signal to next neuron
- Synapse: connection point — can strengthen or weaken (learning!)
- Fires if total input exceeds a threshold

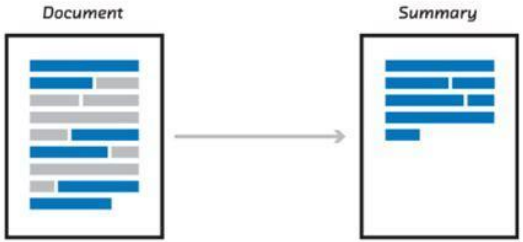


Artificial Neuron

- Inputs $x_1, x_2, ..., x_n$ (like dendrites)
- Weights $w_1, w_2, ..., w_n$ (synapse strength)
- Weighted sum: $z = \Sigma w_i x_i + b$ (bias)
- Activation function: output = f(z)
- Output passed to next layer's neurons
- Learning = adjusting weights (like synapse strengthening)



Examples of end-to-end deep learning



Document summarization

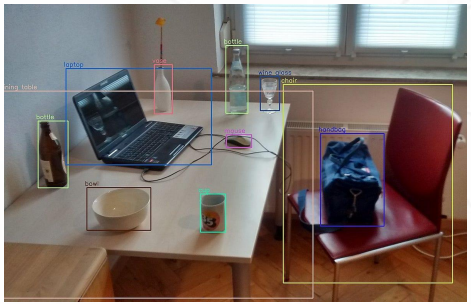
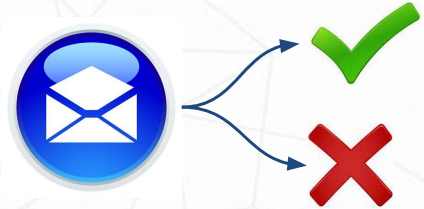


Image segmentation

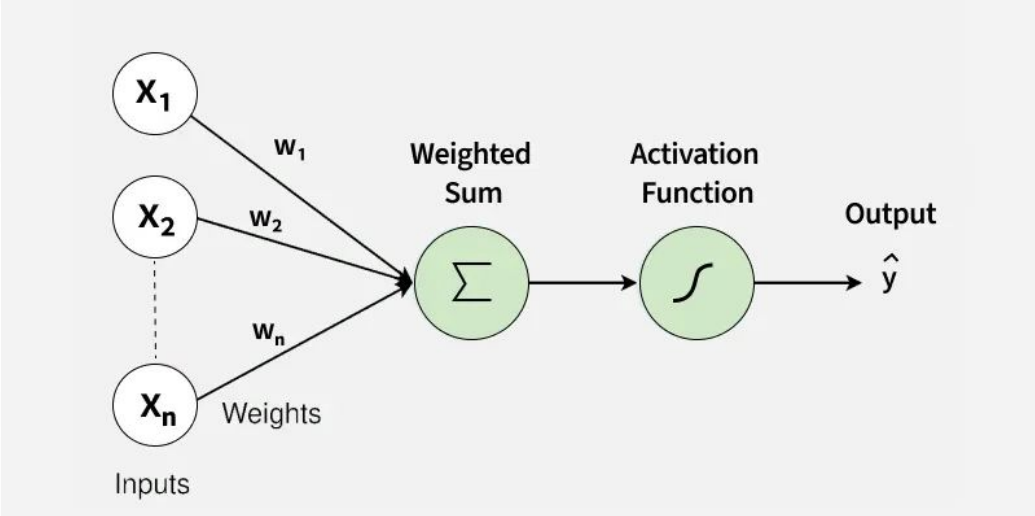


Spam detection



Caption generation

The Perceptron — Rosenblatt (1958)



$$z = w_1x_1 + w_2x_2 + w_3x_3 + b = \Sigma\, w_ix_i + b$$

$\hat{y} = f(z)$ where f is an activation function (step, sigmoid, ReLU...)

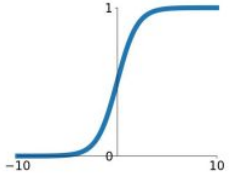
Learning: Adjust w and b to minimize prediction error | Perceptron can only learn linearly separable problems

Activation Functions — Adding Non-Linearity

Activation functions

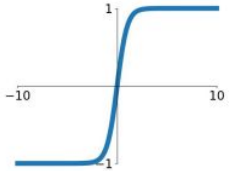
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



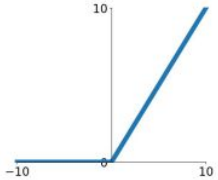
tanh

$$\tanh(x)$$



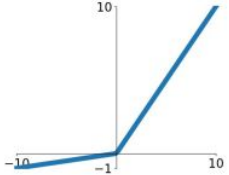
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

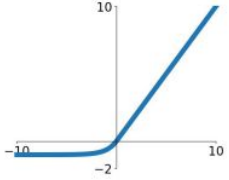


Maxout

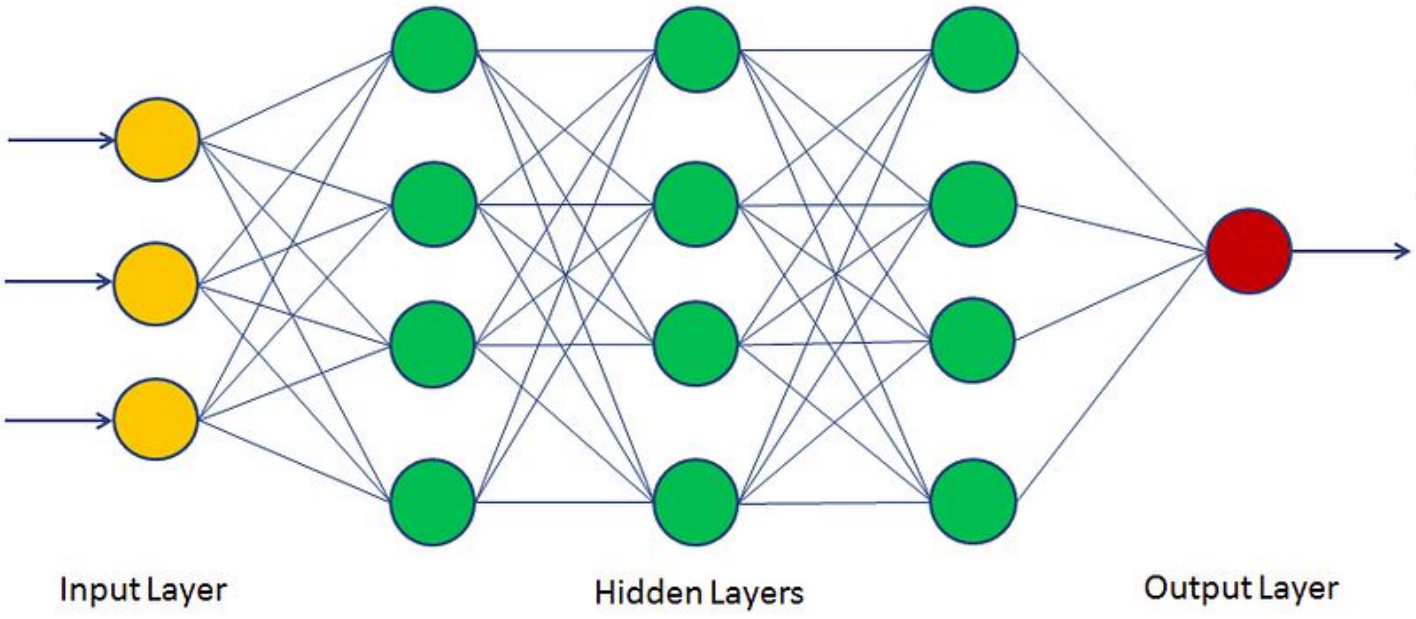
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



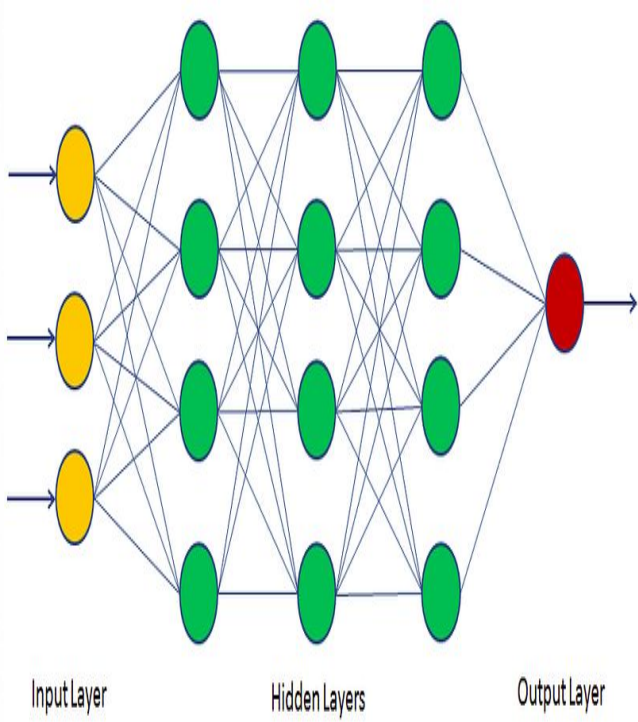
Multi-Layer Perceptron (MLP) — Deep Neural Network



Information flows from left (input) → through hidden layers (feature learning) → to output (prediction).
Each connection has a learnable weight.

Backpropagation — How Neural Networks Learn

- Forward Pass: Input flows through network → produces prediction $\hat{y}$
- Loss Function: Measure error between $\hat{y}$ and true label y
  - MSE for regression: $L = (1/n)\Sigma(y_i - \hat{y}_i)^2$
  - Cross-entropy for classification: $L = -\Sigma\, y_i \log(\hat{y}_i)$
- Backward Pass: Compute gradient of loss w.r.t. each weight using chain rule
  - $\partial L/\partial w = \partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial z \cdot \partial z/\partial w$ (chain rule, layer by layer)
- Gradient Descent Update: $w \leftarrow w - \eta \cdot \partial L/\partial w$ (η = learning rate)
- Variants: SGD (stochastic), Mini-batch GD, Adam, RMSProp — modern optimizers
- Key challenges: Vanishing gradients (deep nets), overfitting, local minima

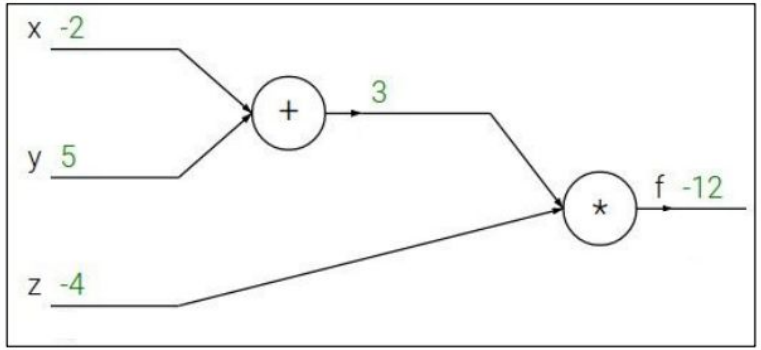


Backpropagation — How Neural Networks Learn

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4



Slides taken from “CS231n: Deep Learning for Computer Vision Stanford”

Backpropagation — How Neural Networks Learn

Backpropagation: a simple example

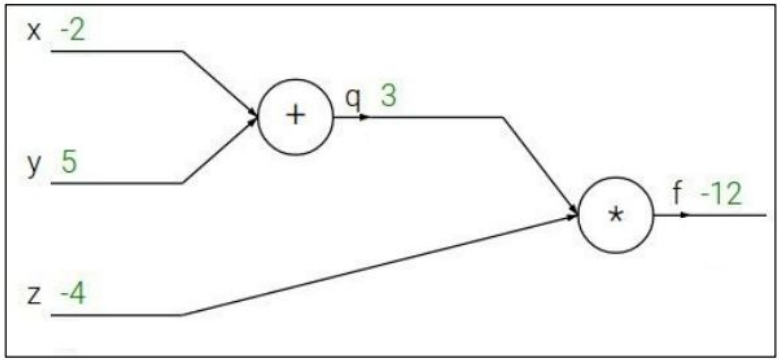
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Slides taken from “CS231n: Deep Learning for Computer Vision Stanford”

Backpropagation — How Neural Networks Learn

Backpropagation: a simple example

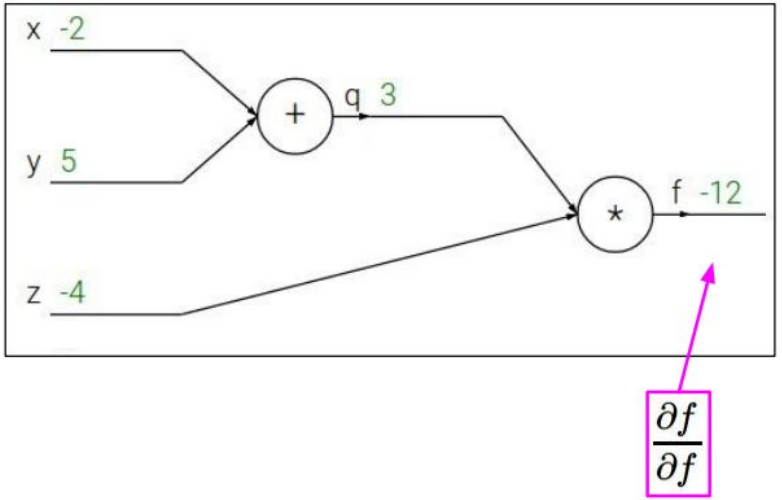
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Slides taken from “CS231n: Deep Learning for Computer Vision Stanford”

Backpropagation — How Neural Networks Learn

Backpropagation: a simple example

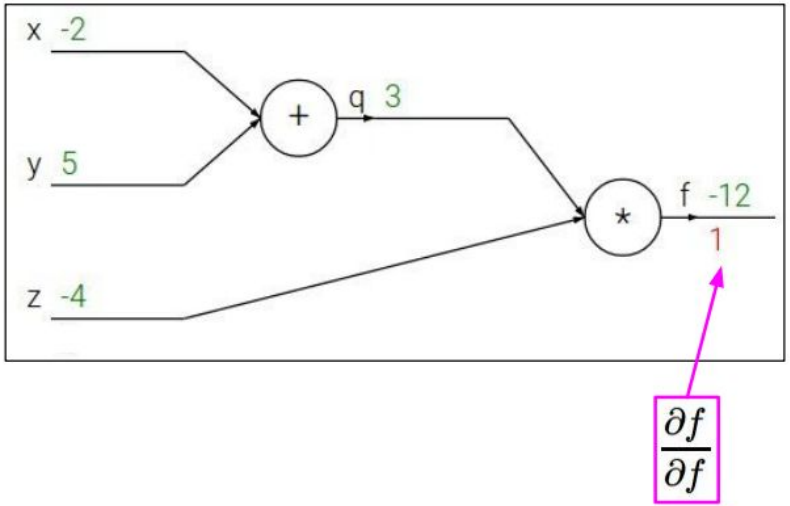
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Slides taken from “CS231n: Deep Learning for Computer Vision Stanford”

Backpropagation — How Neural Networks Learn

Backpropagation: a simple example

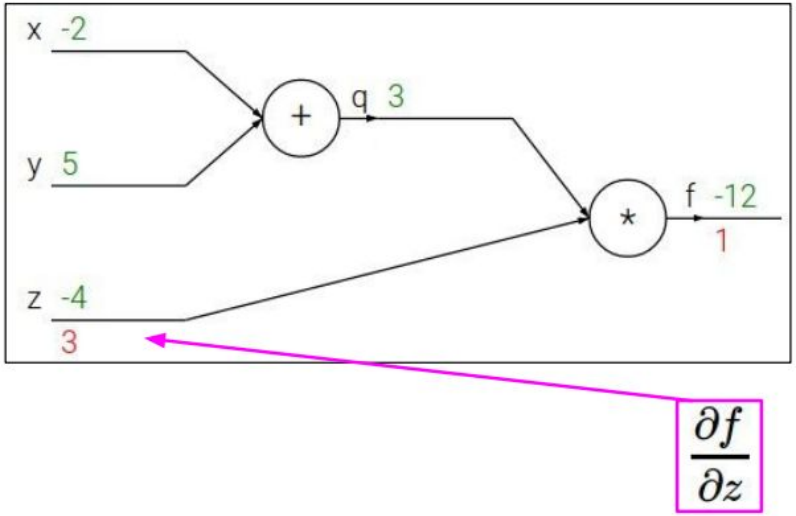
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Slides taken from “CS231n: Deep Learning for Computer Vision Stanford”

Backpropagation — How Neural Networks Learn

Backpropagation: a simple example

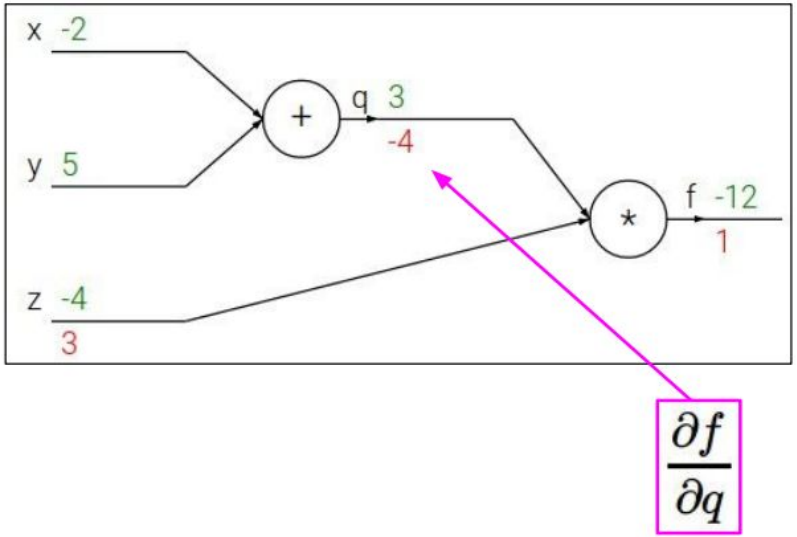
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Slides taken from “CS231n: Deep Learning for Computer Vision Stanford”

Backpropagation — How Neural Networks Learn

Backpropagation: a simple example

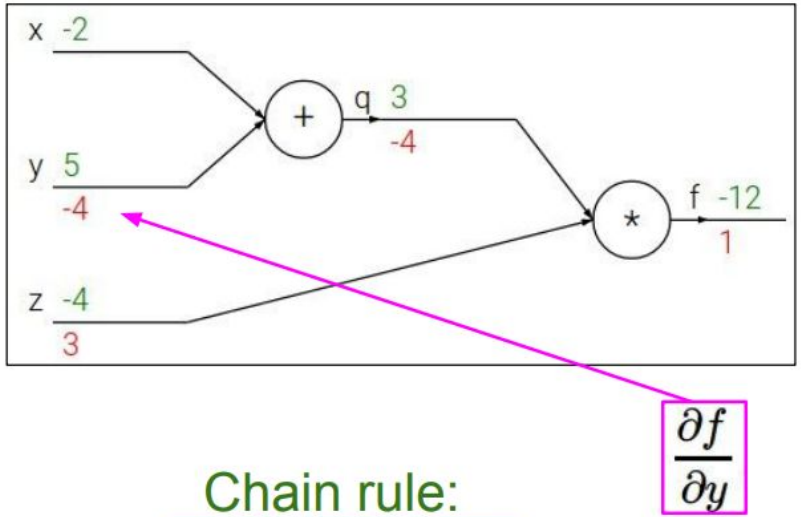
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Slides taken from “CS231n: Deep Learning for Computer Vision Stanford”

Backpropagation — How Neural Networks Learn

Backpropagation: a simple example

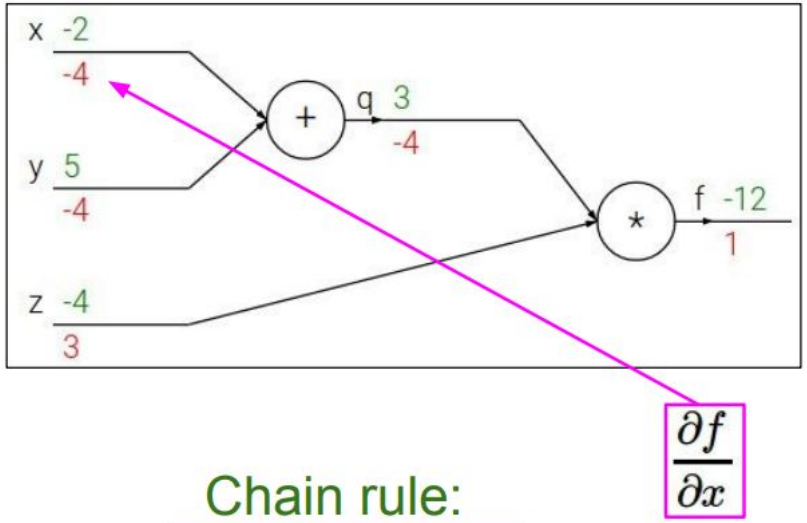
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$$

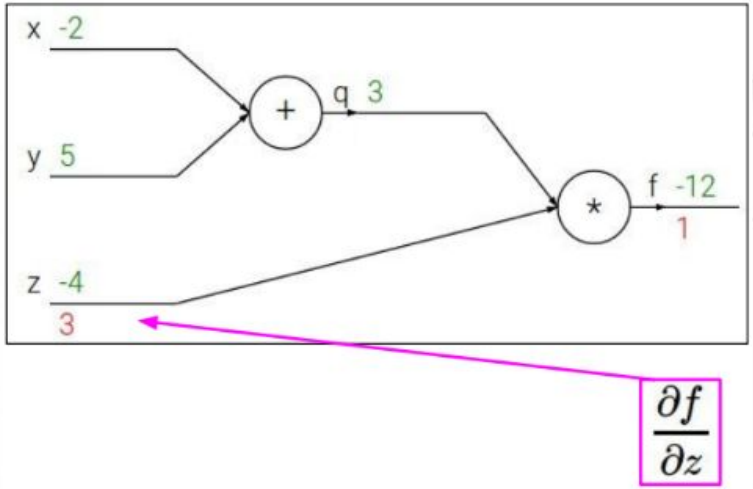
Slides taken from “CS231n: Deep Learning for Computer Vision Stanford”

Backpropagation — How Neural Networks Learn

If we consider the weight *z* and learning rate as 0.1, then the updated value of z,

$$z := z - \eta \frac{\partial f}{\partial z}$$

$$z := -4 - ((0.1) * (3)) = -4.3$$



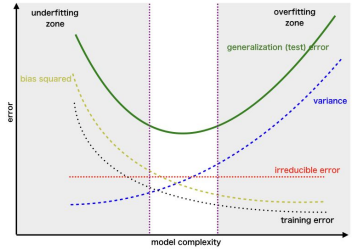
💡 **Key Insight:** The weight z is nudged slightly opposite to the gradient, reducing the loss with each step.

Slides taken from “CS231n: Deep Learning for Computer Vision Stanford”

Overfitting, Underfitting & Regularization

Bias-Variance Tradeoff

- Underfitting (High Bias): Model too simple, cannot capture patterns
- Overfitting (High Variance): Model memorizes training data, fails on new data
- The sweet spot: Generalizes well to unseen data
- Monitored via Train/Validation/Test split
- Training accuracy vs Validation accuracy gap = overfitting signal



Regularization Techniques

- L2 Regularization (Weight Decay): Penalize large weights in loss function
- L1 Regularization: Induces sparsity (feature selection)
- Dropout: Randomly zero out neurons during training (prevents co-adaptation)
- Batch Normalization: Normalize layer activations for stable training
- Data Augmentation: Expand training set with transforms (flip, crop, noise)
- Early Stopping: Stop training when validation loss starts increasing

What Are Overfitting & Underfitting?



Underfitting

Model too simple — misses patterns in training data. High bias, low variance. Degree-1 poly on curved data.

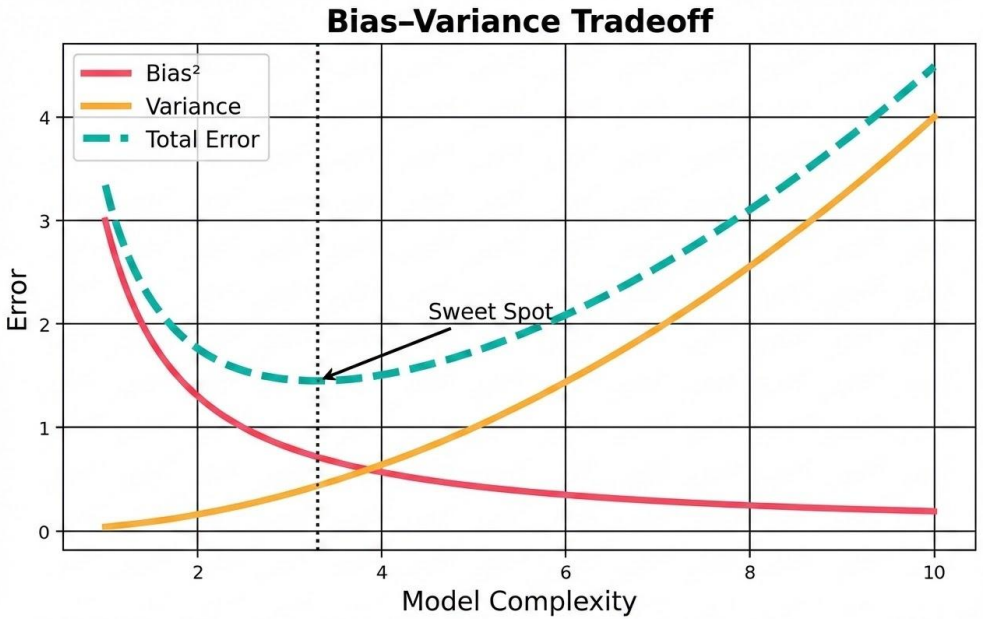
Good Fit

Captures the true pattern without memorising noise. Low bias + low variance. The goal of every model.

Overfitting

Model too complex — memorises noise. Low train error, high val error. High variance, low bias.

The Bias–Variance Tradeoff



High Bias

Underfitting. Model assumptions too rigid. Increases training error.

High Variance

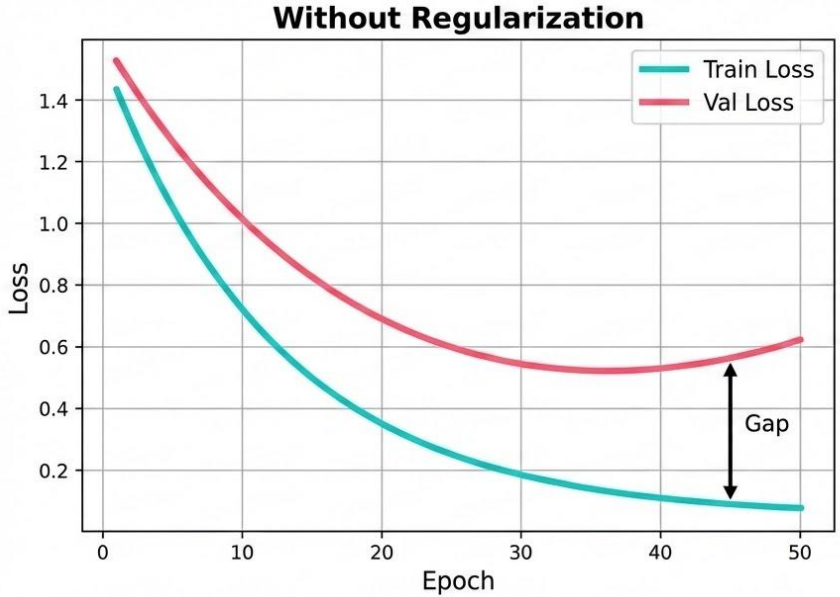
Overfitting. Model too sensitive to training data. Val error diverges.

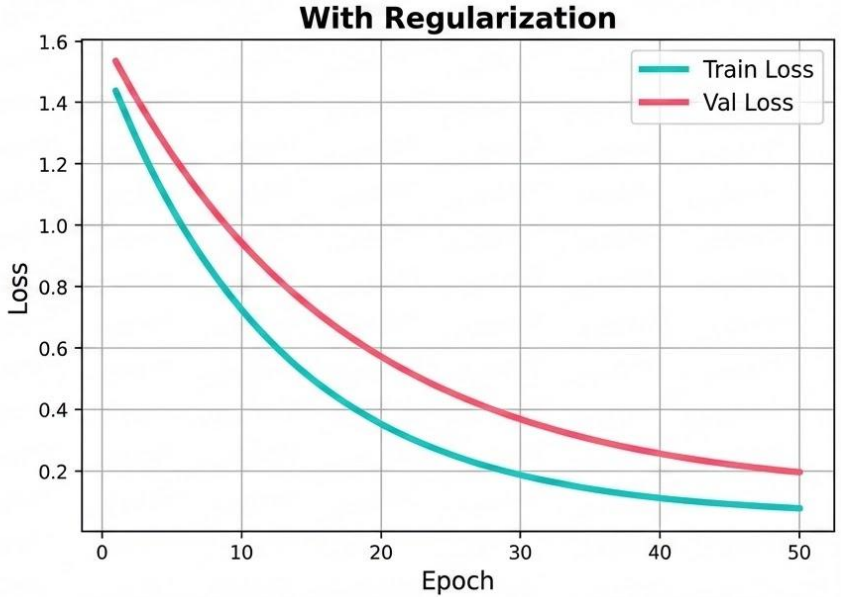
Sweet Spot

Minimise total error = Bias² + Variance + irreducible noise.

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \varepsilon$$

Detecting Overfitting — Training vs Validation Loss





Train loss ↓ but val loss ↑ → overfitting. Stop training early or add regularization.

With regularization, val loss tracks train loss — the gap shrinks significantly.

Both losses staying high → underfitting. Increase model capacity or train longer.

Regularization — Penalise Complexity

What is Regularization?

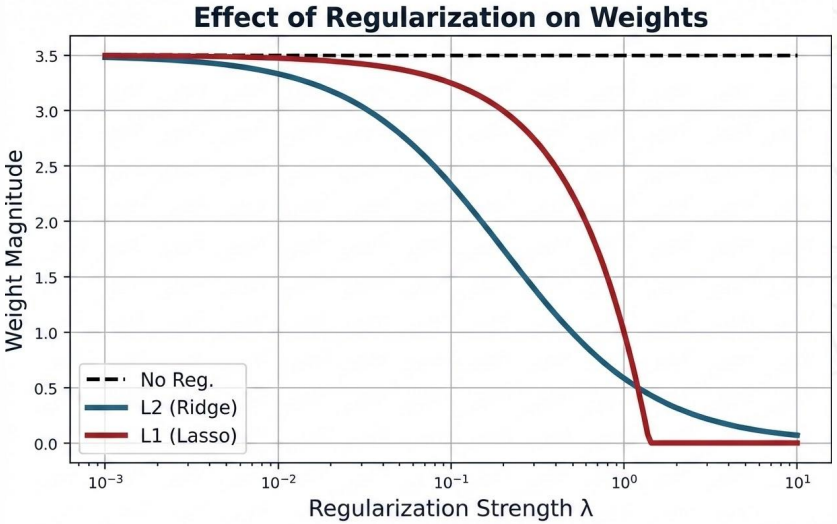
Adds a penalty term to the loss function to discourage large weights. Forces the model to stay simple and generalize better.

L2 (Ridge)

Loss + $\lambda \cdot \Sigma w^2$ Shrinks all weights smoothly toward zero.

L1 (Lasso)

Loss + $\lambda \cdot \Sigma |w|$ Drives some weights exactly to zero (feature selection).




```
# Sklearn: Ridge (L2) & Lasso (L1)
from sklearn.linear_model import Ridge, Lasso

ridge = Ridge(alpha=1.0)    # alpha = λ
ridge.fit(X_train, y_train)

lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
```

L1 vs L2 — Geometry & Sparsity

L1 — Lasso

Constraint: diamond shape

- Penalty: $\lambda \cdot \Sigma|w_i|$
- Creates sparse models (weights → 0)
- Good for feature selection
- Non-differentiable at 0
- Solver: coordinate descent

```
# L1 pushes weights to zero
lasso = Lasso(alpha=0.5)
print(lasso.coef_)
# → [0. 1.23 0. 0. 0.87]
```

L2 — Ridge

Constraint: circle shape

- Penalty: $\lambda \cdot \Sigma w_i^2$
- Shrinks all weights uniformly
- No exact zeros — keeps all features
- Differentiable everywhere
- Has analytical closed-form solution

```
# L2 shrinks weights evenly
ridge = Ridge(alpha=1.0)
print(ridge.coef_)
# → [0.12 0.88 0.03 0.21 0.75]
```

More Regularization Techniques — Dropout & Early Stopping

Dropout

Randomly zero-out neurons during training (rate p). Forces network to learn redundant representations and prevents co-adaptation.

```
import torch.nn as nn

model = nn.Sequential(
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Dropout(p=0.5),   # 50% dropout
    nn.Linear(64, 10)
)
```

Early Stopping

Monitor validation loss. Stop training when it stops improving (patience window). Keeps the best checkpoint automatically.

```
from tensorflow.keras.callbacks import EarlyStopping

cb = EarlyStopping(
    monitor='val_loss',
    patience=5,         # wait 5 epochs
    restore_best_weights=True
)

model.fit(X, y, callbacks=[cb])
```

Putting It Together — Sklearn Pipeline

```
#import packages
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import (
   PolynomialFeatures, StandardScaler)
from sklearn.linear_model import Ridge
from sklearn.model_selection import (
   train_test_split, cross_val_score)

np.random.seed(42)
X = np.sort(np.random.rand(80,1), axis=0)
y = np.sin(2*np.pi*X).ravel() +     np.random.normal(0, 0.15, 80)
X_train, X_test, y_train, y_test =     train_test_split(X, y,
test_size=0.2)
```

```
# Regularized polynomial pipeline
pipe = Pipeline([
   ('poly',   PolynomialFeatures(degree=9)),
   ('scaler', StandardScaler()),
   ('model',  Ridge(alpha=1.0))   # L2
])
pipe.fit(X_train, y_train)

train_r2 = pipe.score(X_train, y_train)
test_r2  = pipe.score(X_test,  y_test)
print(f"Train R²: {train_r2:.3f}")
print(f"Test  R²: {test_r2:.3f}")

# Cross-validation
cv = cross_val_score(pipe, X, y, cv=5)
print(f"CV: {cv.mean():.3f} ± {cv.std():.3f}")
```

✔ Always hold out a test set — never tune on it.

✔ Use cross-validation to get stable performance estimates.

✔ Tune α (lambda) with GridSearchCV or validation curve.

Summary — Quick Reference

Problem	Symptom	Fix	Code Lever
Underfitting	High train + val error	Bigger model, more features, more epochs	degree=↑, layers=↑, epochs=↑
Overfitting	Low train, high val error	Regularize, dropout, more data	Ridge/Lasso alpha=↑, Dropout(p)
High Variance	Train >> Val score	L2 penalty, cross-validate	Ridge(alpha), cross_val_score
High Bias	Both scores low	Add polynomial features, tune model class	PolynomialFeatures(degree=↑)

Thank you

Instructor



Niloy Ganguly
niloy@cse.iitkgp.ac.in

Teaching Assistants (IITKGP)



Sombit Bose



Shrimon Mukherjee